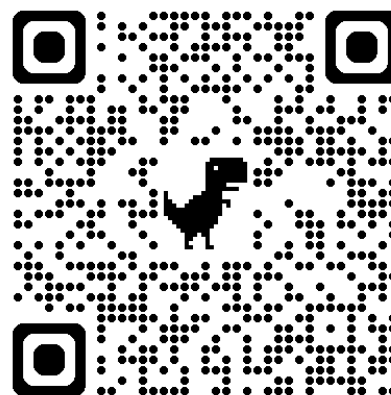




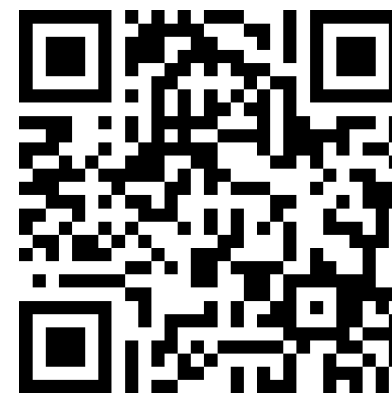
深度學習 Deep Learning

神經網路與梯度下降

Instructor: 林英嘉 (Ying-Jia Lin)
2026/09/15



[Course GitHub](#)



[Slido # DL0915](#)

Outline

- Multi-layer perceptron (MLP)
 - 基礎神經網路
- 權重值如何被更新 (梯度下降)

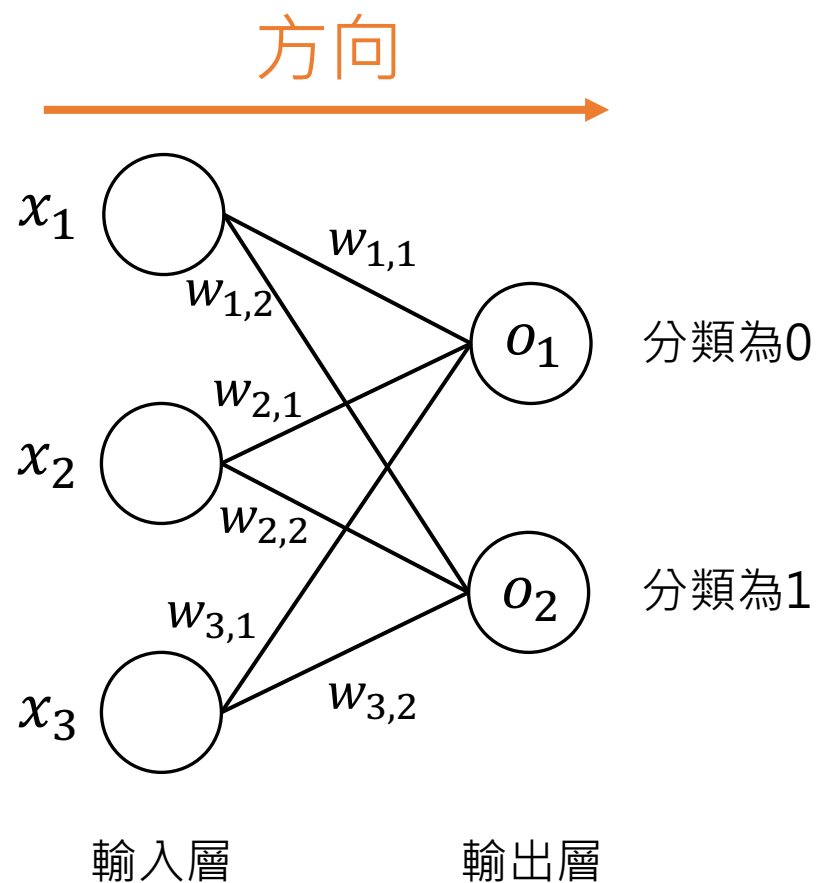


[Recap] 什麼是深度學習？

- 深度學習是一種機器學習方法，著重於使用多層的類神經網路 (Neural Networks) 來完成任務，例如分類、生成等等。
- 在深度學習模型中，通常參數數量都非常龐大，這主要來自於隱藏層大小 (hidden size) 與模型層數 (number of hidden layers) 的增加。



[Recap] 簡單的類神經網路：Perceptron (感知機)



x 代表輸入
 w 代表權重值

代表 x 有多少比例
要被傳至下一層

今日重點

$$o_1 = x_1w_{1,1} + x_2w_{2,1} + x_3w_{3,1}$$

只有經過一層權重值計算



Introduction to Matrix (矩陣)

- 矩陣：矩形陣列
- M 個 rows (列) x n 個 columns (行)
 - (程式中) 矩陣形狀為：(m, n)
 - 矩陣形狀又稱 shape, size

$$\begin{array}{c} \text{rows} \end{array} \begin{array}{c} \text{columns} \end{array} \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$



Matrix Multiplication

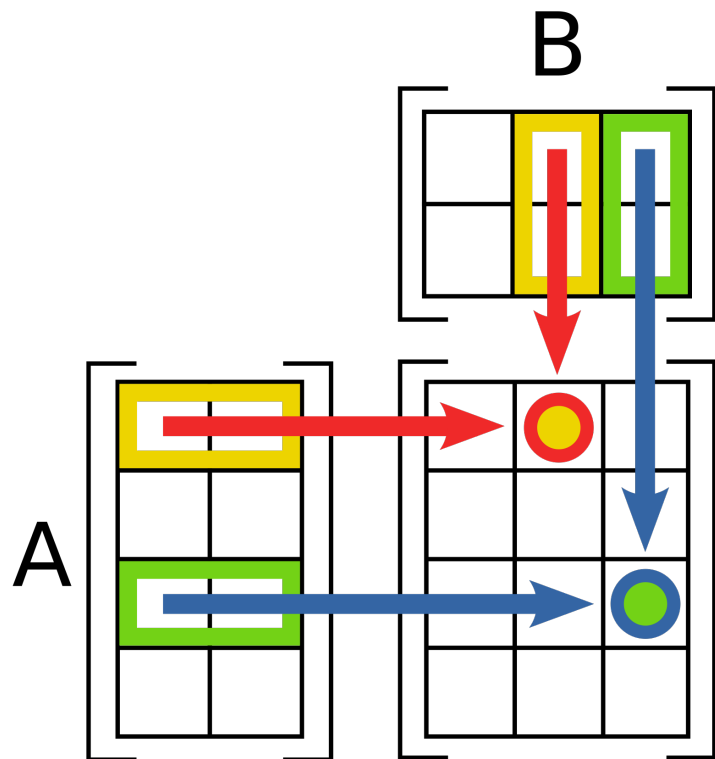
- 假設有兩矩陣A和B，形狀分別為 (m, n) 跟 (n, p) ，

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \dots & \color{red}{a_{1,n}} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix} \quad \begin{bmatrix} \color{red}{b_{1,1}} & b_{1,2} & \dots & b_{1,p} \\ b_{2,1} & b_{2,2} & \dots & b_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ \color{red}{b_{n,1}} & b_{n,2} & \dots & b_{n,p} \end{bmatrix}$$

$$\begin{aligned} \text{則 } AB &= a_{1,1} \times b_{1,1} + a_{1,2} \times b_{2,1} + \dots + \color{red}{a_{1,n}} \times \color{red}{b_{n,1}} + \\ &\quad a_{2,1} \times b_{1,2} + a_{2,2} \times b_{2,2} + \dots + a_{2,n} \times b_{n,2} + \dots \\ &\quad + a_{m,1} \times b_{1,p} + a_{m,2} \times b_{2,p} + \dots + a_{m,n} \times b_{n,p} \end{aligned}$$



Matrix Multiplication



Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 10 \\ 25 \end{bmatrix}$$

https://zh.wikipedia.org/zh-tw/矩陣乘法#/media/File:Matrix_multiplication_diagram.svg



[Recap] 類神經網路：Perceptron (感知機)

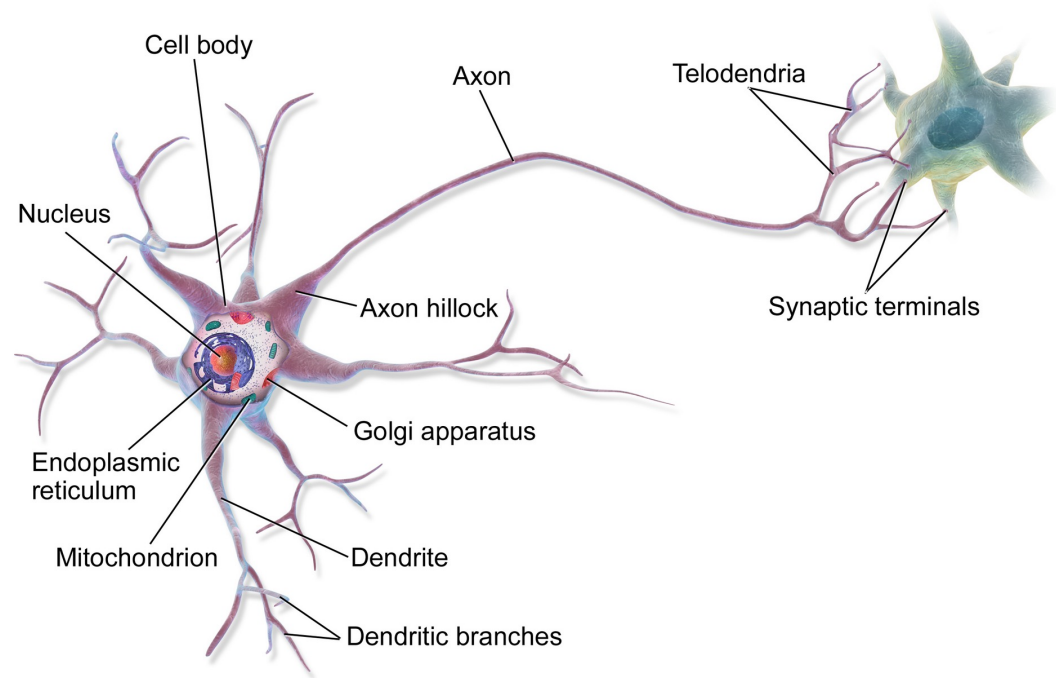
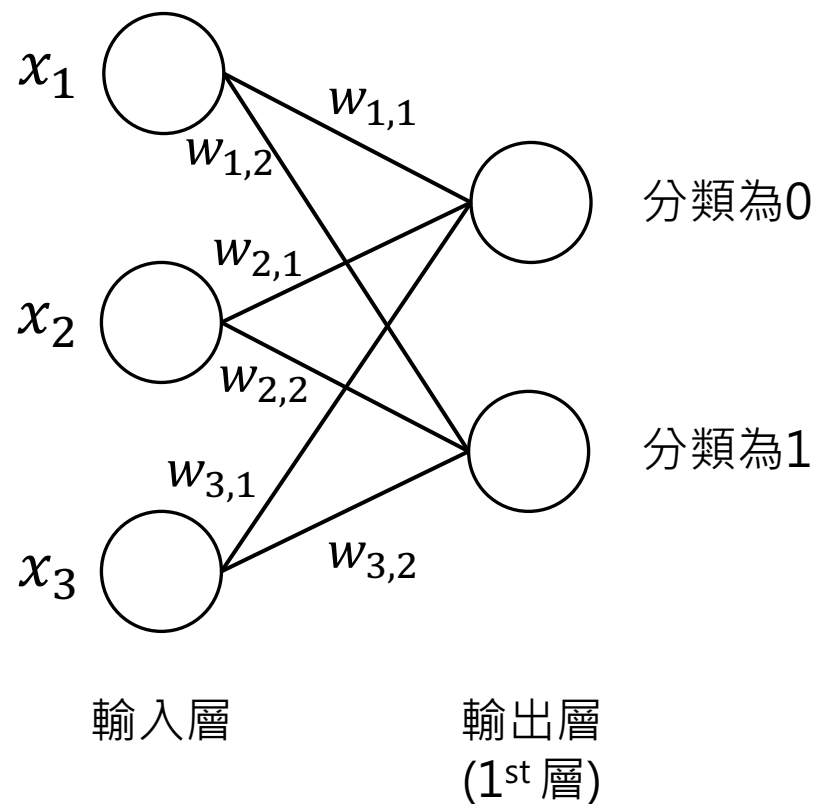
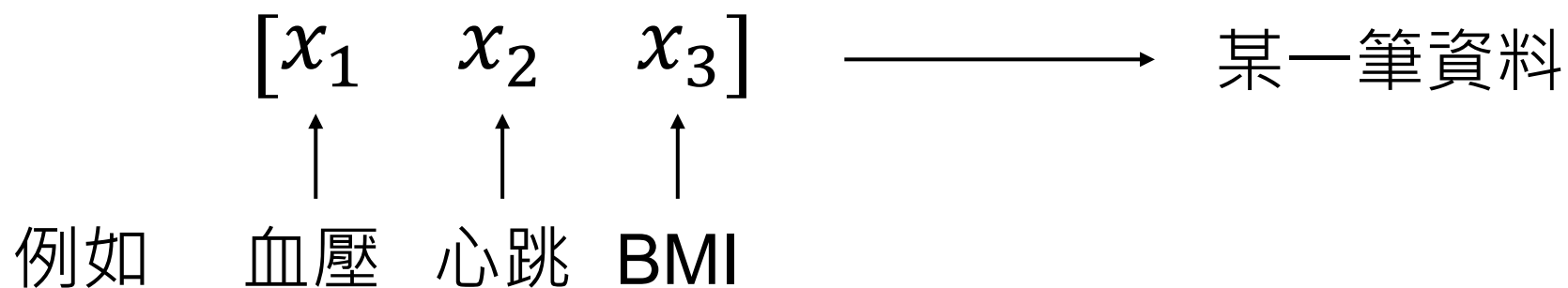


Figure source: <https://zh-yue.wikipedia.org/wiki/神經細胞>



模型 (網路) 的輸入是什麼？

數值型資料



影像資料

影像像素值 (pixel)

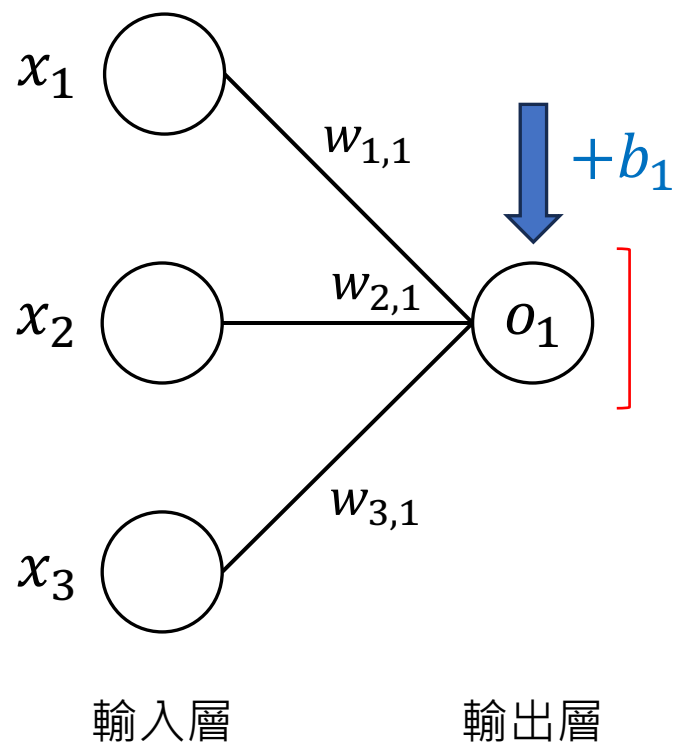


$$\longrightarrow \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$



Math for Perceptron using a toy example (1)

output_size = 1



$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \times \begin{bmatrix} w_{1,1} \\ w_{2,1} \\ w_{3,1} \end{bmatrix} + [b_1] = [o_1]$$

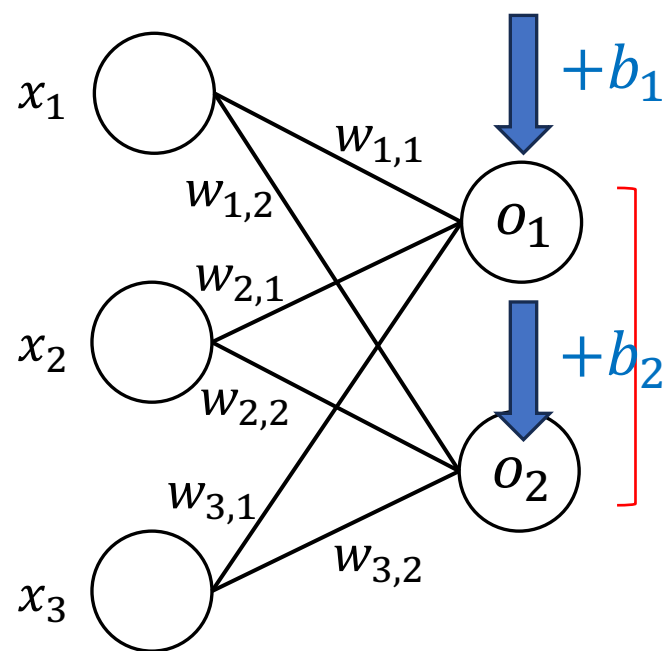
$\mathbb{R}^{1 \times 3} \quad \mathbb{R}^{3 \times 1} \quad \mathbb{R}^1 \quad \mathbb{R}^1$

bias



Math for Perceptron using a toy example (2)

output_size = 2



輸入層

輸出層
(1st層)

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \times \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 \end{bmatrix} = \begin{bmatrix} o_1 & o_2 \end{bmatrix}$$

$\mathbb{R}^{1 \times 3}$ $\mathbb{R}^{3 \times 2}$ $\mathbb{R}^{1 \times 2}$ $\mathbb{R}^{1 \times 2}$

bias

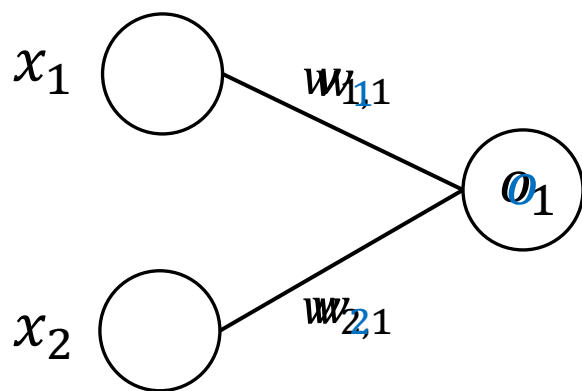


bias 的定義與說明

- bias (term): 偏置項
- 偏差值幫助將學到的函數平移
- bias 也是可調整的參數
 - 因此，bias 通常能使分類效果更好

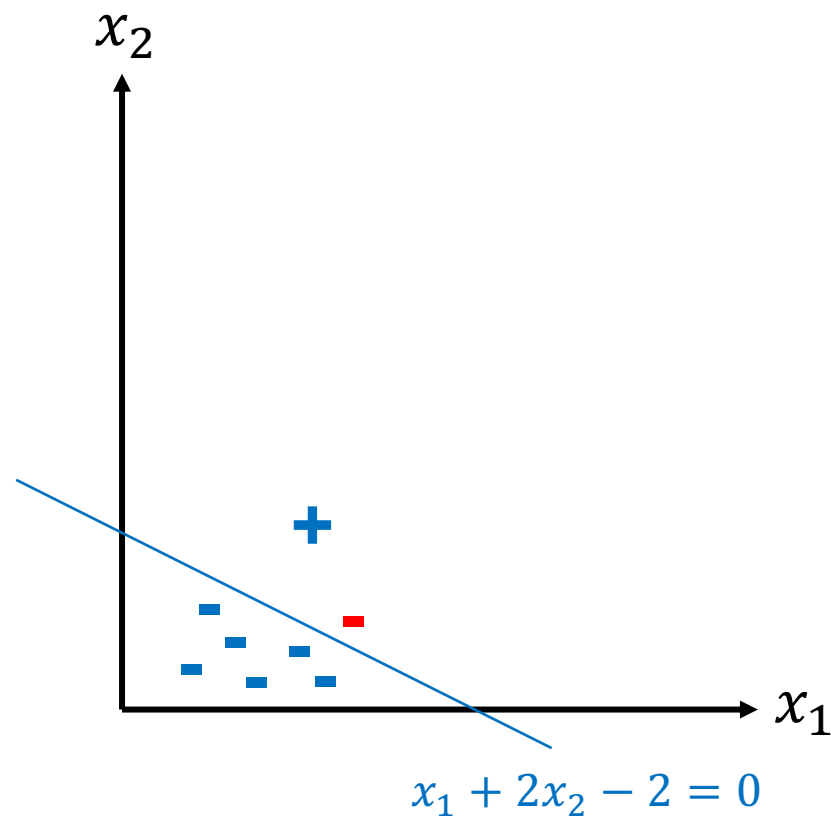


感知機與直線方程式

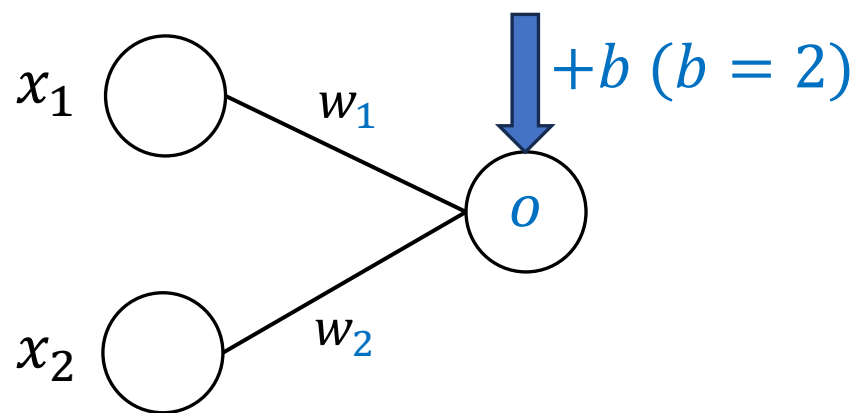


$$o_1 = x_1 w_{1,1} + x_2 w_{2,1}$$

$$w_1 = 1, w_2 = 2, o = 2$$

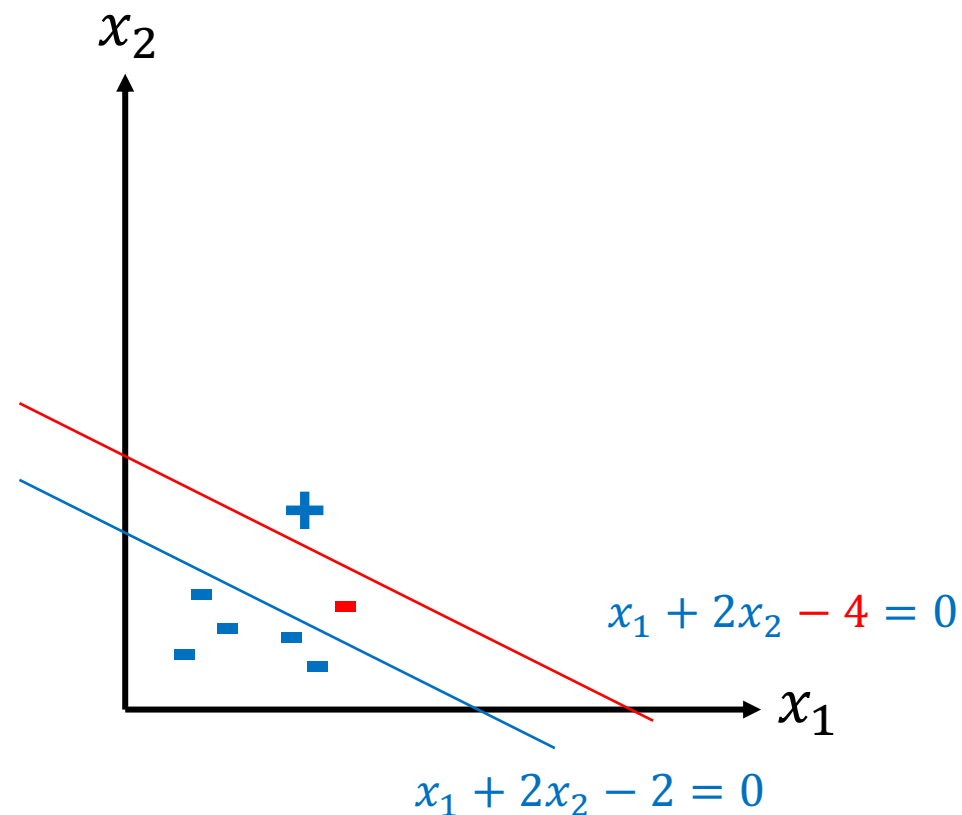


感知機與直線方程式 (結合 bias)



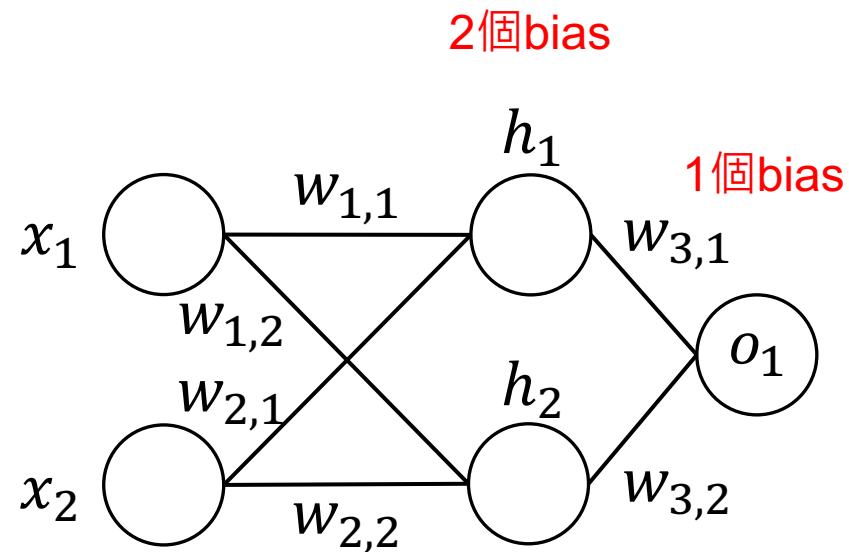
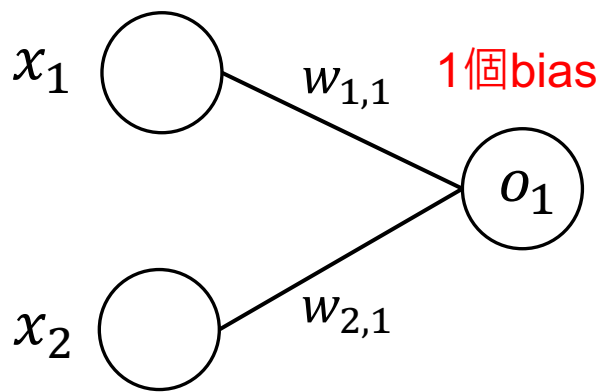
$$o = x_1 w_1 + x_2 w_2$$

$$w_1 = 1, w_2 = 2, o = 2$$



bias 數量

- bias 數量等同於該層的輸出維度，因為每個神經元都需要一個偏移量，來調整自己的**輸出值**
 - hidden layer 就是 hidden size
 - output layer 就是 output size



Perceptron 小結

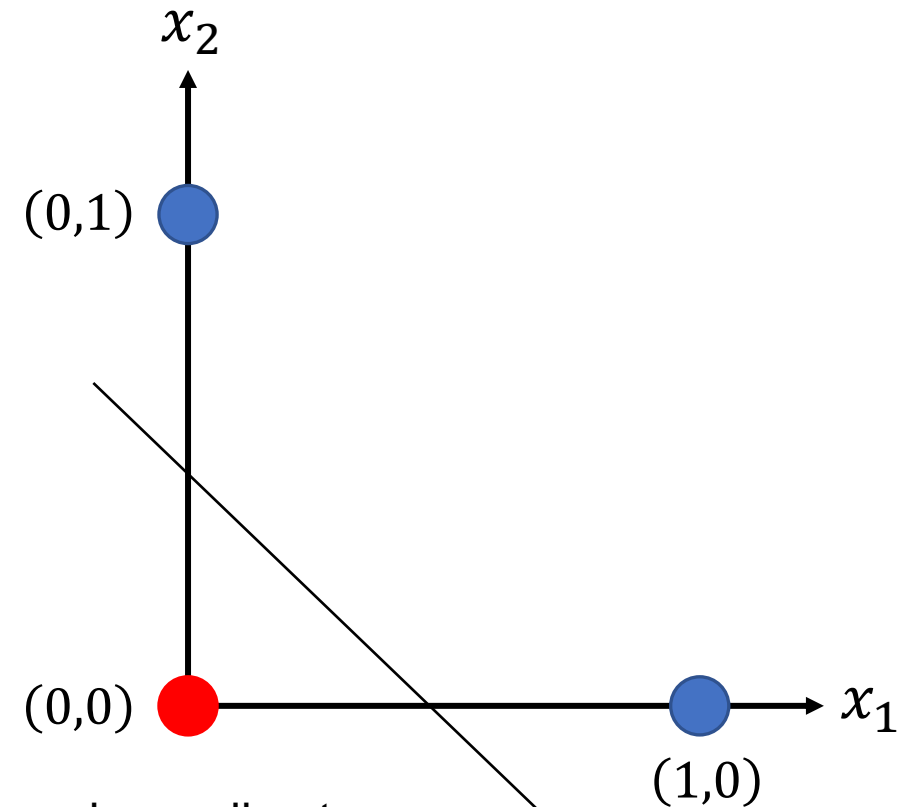
- 權重對於輸入的矩陣乘法的過程相當於進行**線性**轉換
- 全連接 (fully-connected)
- bias 數量等同於該層的輸出維度



A toy example for classification

x1	x2	Output
0	0	0
0	1	1
1	0	1

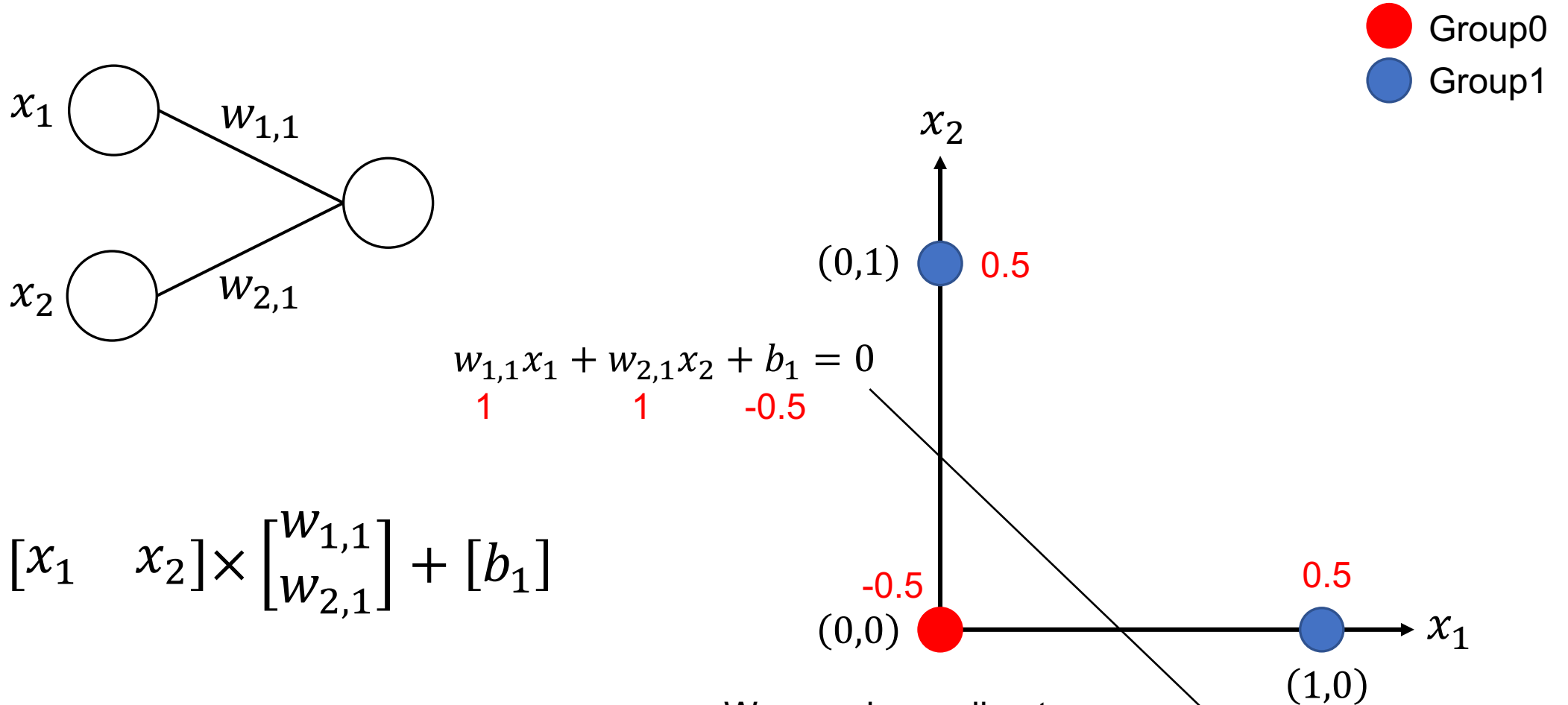
● Group0
● Group1



We can draw a line to
classify two groups!



A toy example for classification with Perceptron



We can draw a line to
classify two groups!



XOR Problem

x1	x2	AND
0	0	0
0	1	0
1	0	0
1	1	1

x1	x2	OR
0	0	0
0	1	1
1	0	1
1	1	1

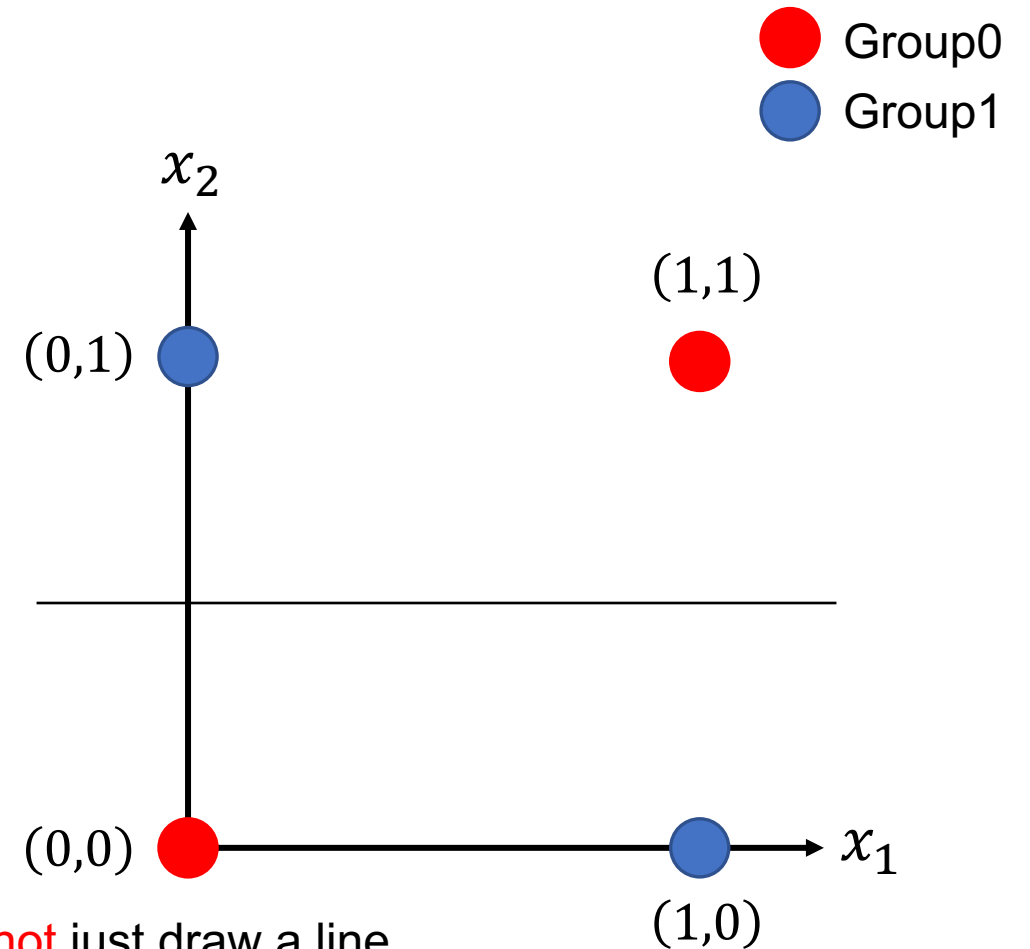
x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



XOR Problem

XOR: Exclusive-or (邏輯算符互斥或)

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



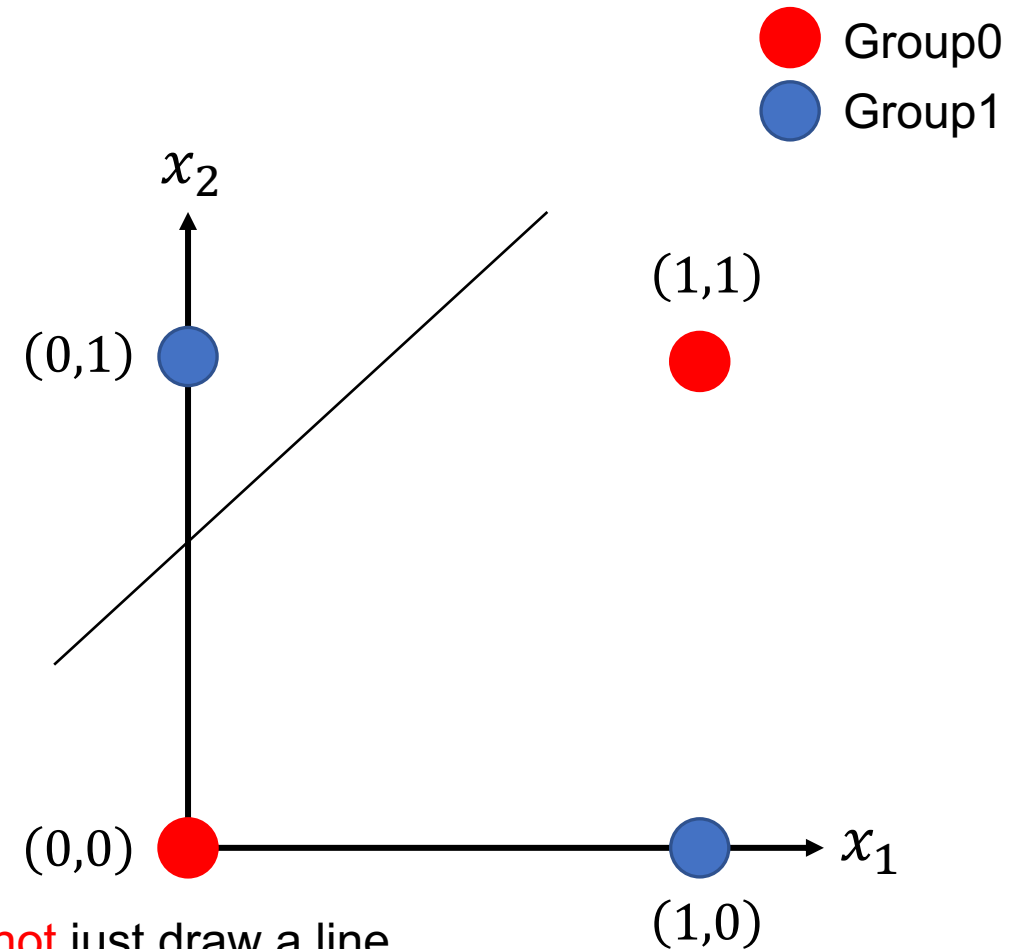
We **cannot** just draw a line to classify two groups!



XOR Problem

XOR: Exclusive-or (邏輯算符互斥或)

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



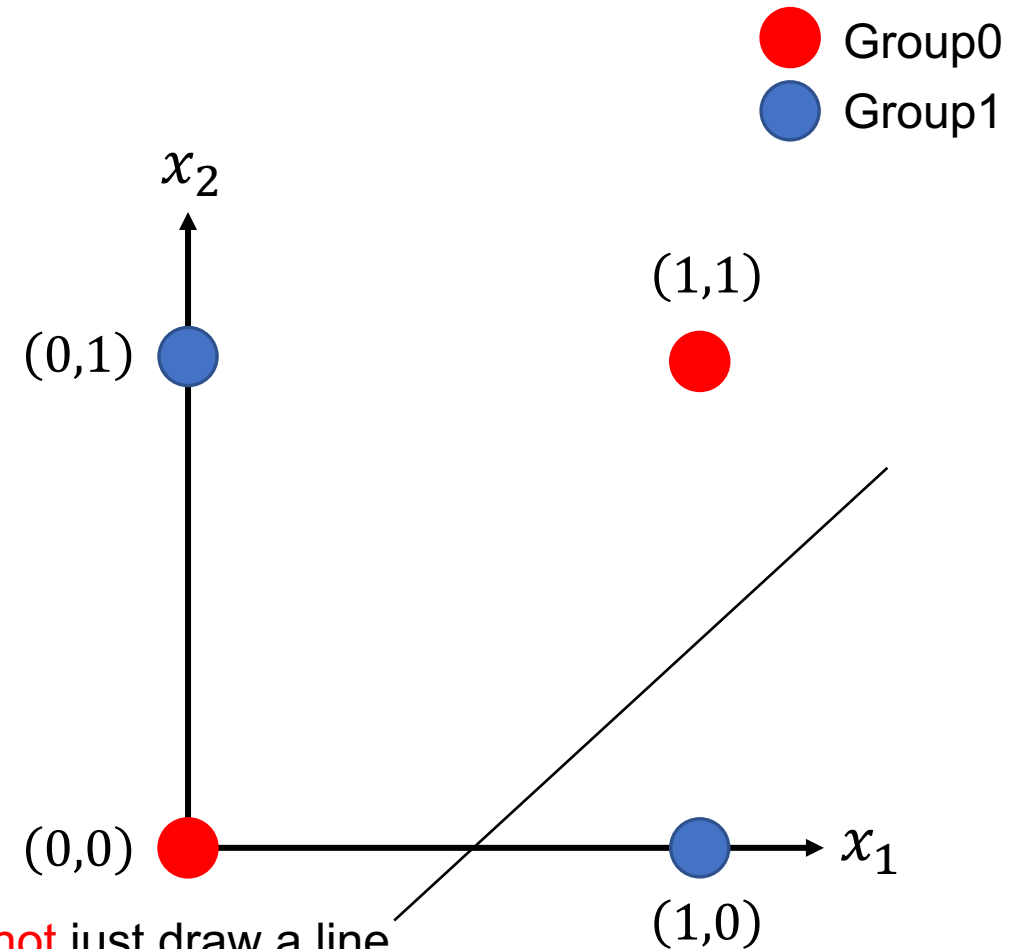
We **cannot** just draw a line to classify two groups!



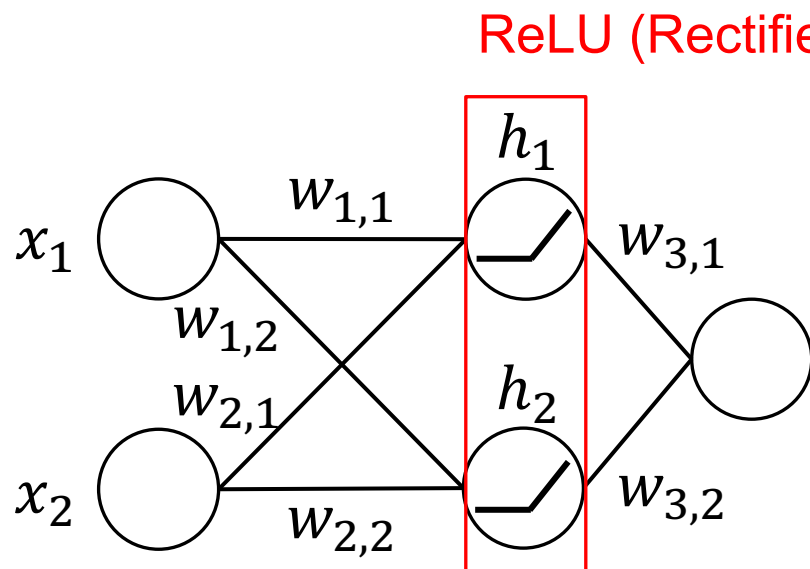
XOR Problem

XOR: Exclusive-or (邏輯算符互斥或)

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0



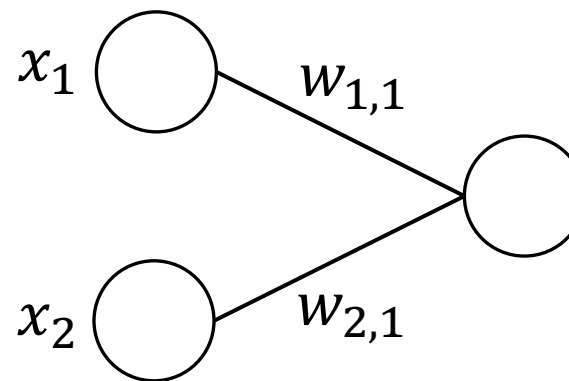
MLP for the XOR Problem



輸入層

隱藏層
(1st 層)

輸出層
(2nd 層)



Perceptron通常沒有隱藏層



ReLU (Rectified Linear Unit)^{[1][2]}

- **Non-linear** transformation
- Negative values will be transformed to zeros.
- Positive values remain their original ones.

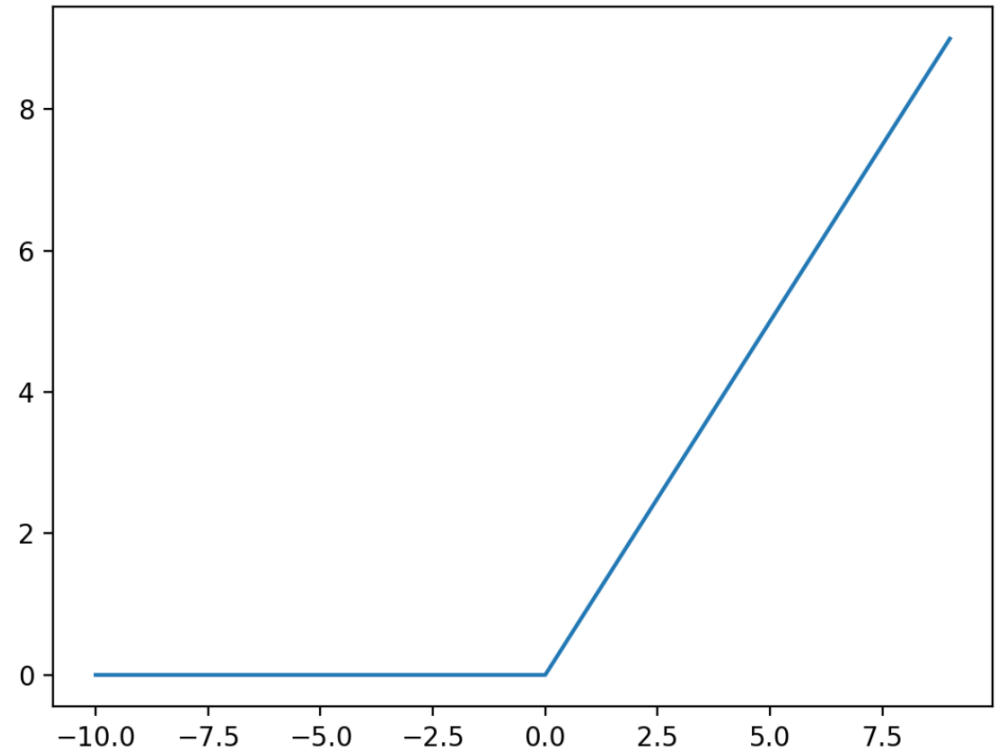


Figure source:
<https://paperswithcode.com/method/relu>



[1] Jarrett, Kevin, et al. "What is the best multi-stage architecture for object recognition?." ICCV 2009.

[2] Nair, Vinod, and Geoffrey E. Hinton. "Rectified linear units improve restricted boltzmann machines." ICML 2010.

為什麼 ReLU 是非線性函數？

線性函數需要有疊加性： $f(x+y) = f(x) + f(y)$ 與齊次性： $f(ax) = a*f(x)$

假設 ReLU 為 f ：

左式： $f(1+(-1)) = f(0) = 0$

右式： $f(1) + f(-1) = 1 + 0 = 1$

左式與右式不符合，故 ReLU 為非線性。

[齊次性]

假設 ReLU 為 f , $a=-10$, $x=-1$ ：

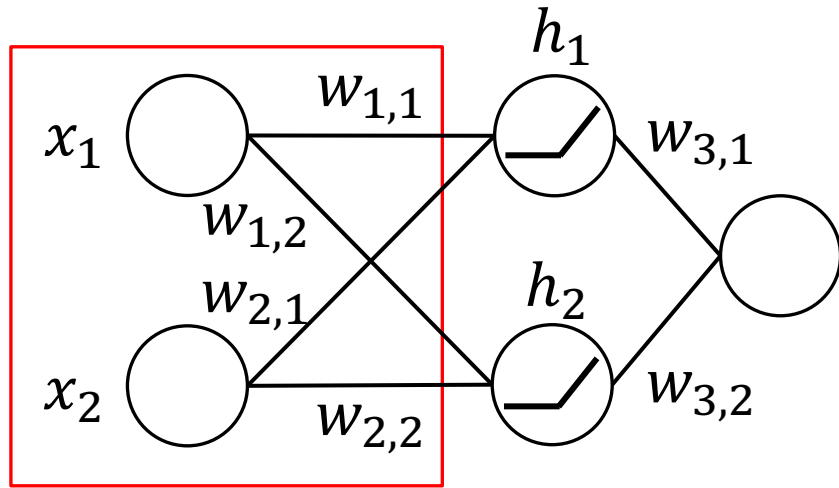
左式： $f(-10 * -1) = f(10) = 10$

右式： $-10 * f(-1) = 0$

左式與右式不符合，故 ReLU 為非線性。



Math of MLP for the XOR Problem (1)



任一筆資料

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \times \begin{bmatrix} w_{1,1}^1 & w_{1,2}^1 \\ w_{2,1}^1 & w_{2,2}^1 \end{bmatrix} + \begin{bmatrix} b_1^0 & b_2^{-1} \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$$

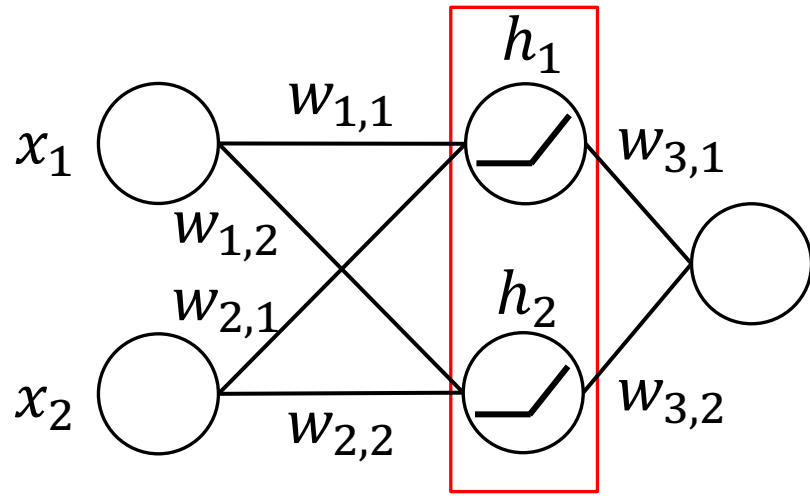
$$\begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & -1 \\ 0 & -1 \\ 0 & -1 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Element-wise
summation with bias

	x1	x2
●	0	0
●	0	1
●	1	0
●	1	1



Math of MLP for the XOR Problem (2)



$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \xrightarrow{\text{ReLU}} \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

ReLU (Rectified Linear Unit)

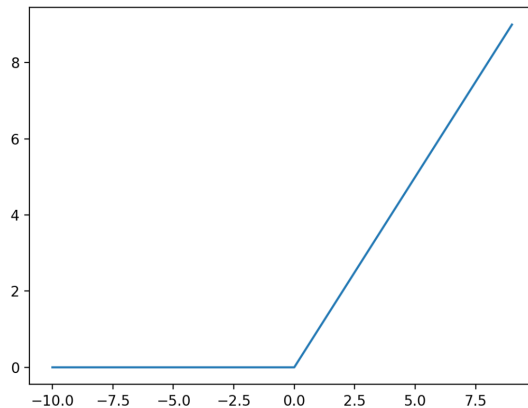
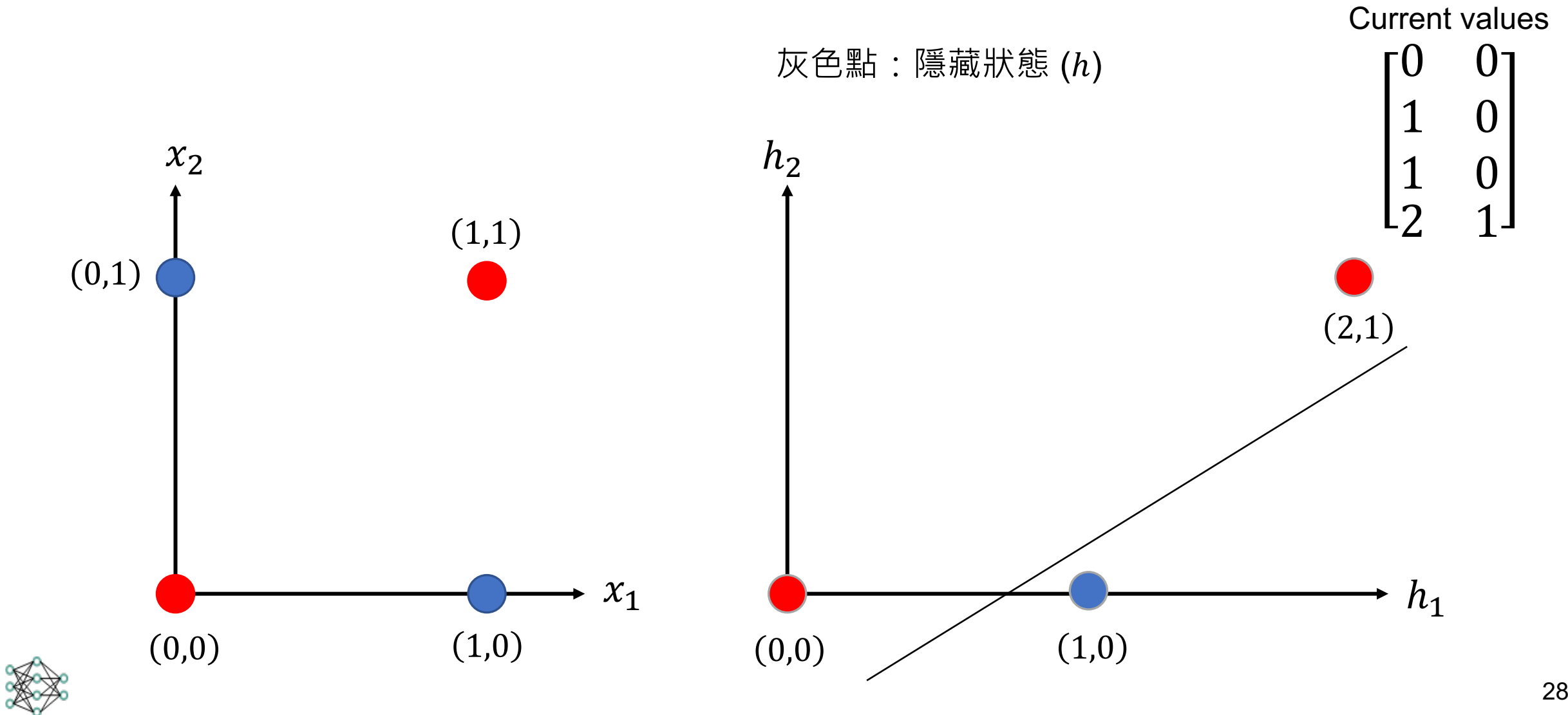


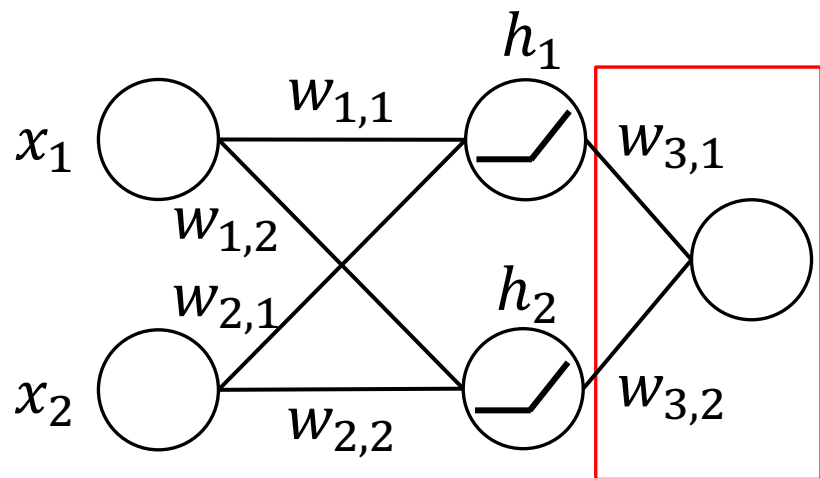
Figure source:
<https://paperswithcode.com/method/relu>



Math of MLP for the XOR Problem (2)



Math of MLP for the XOR Problem (3)



任一筆資料

$$\begin{bmatrix} x_1 & x_2 \end{bmatrix} \times \begin{bmatrix} \overset{1}{w_{3,1}} \\ \underset{-2}{w_{3,2}} \end{bmatrix} + \overset{0}{[b_3]}$$

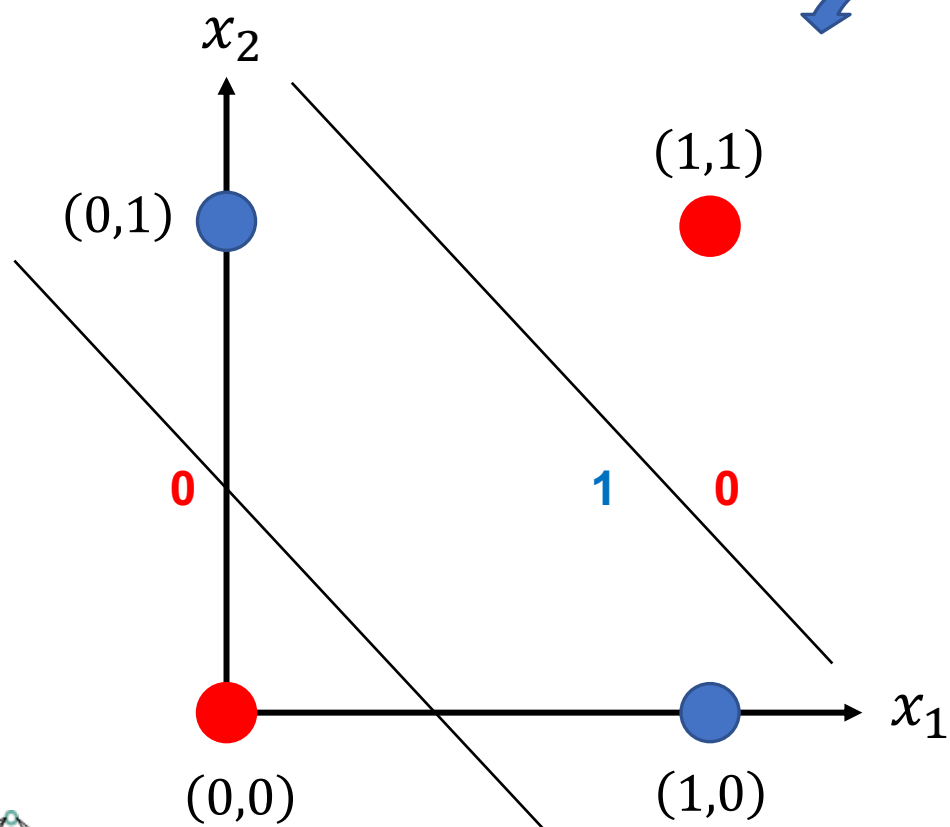
$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

可視為分類層

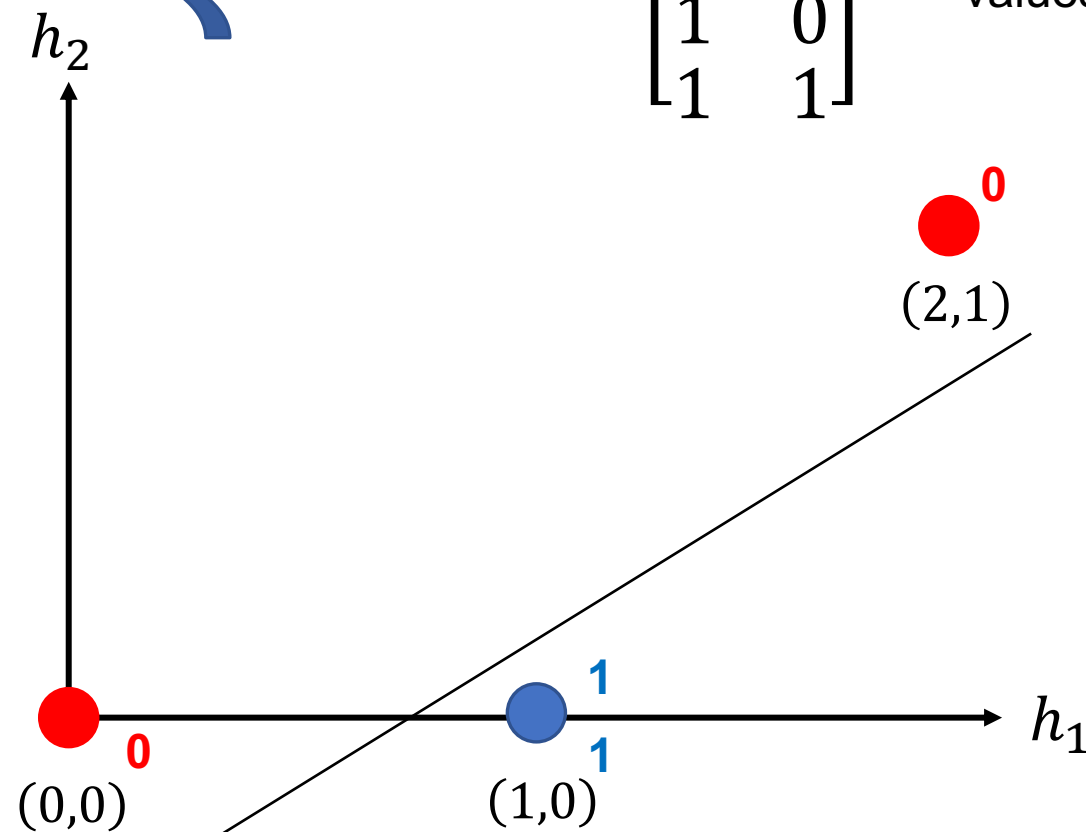


Visualization of MLP for the XOR Problem

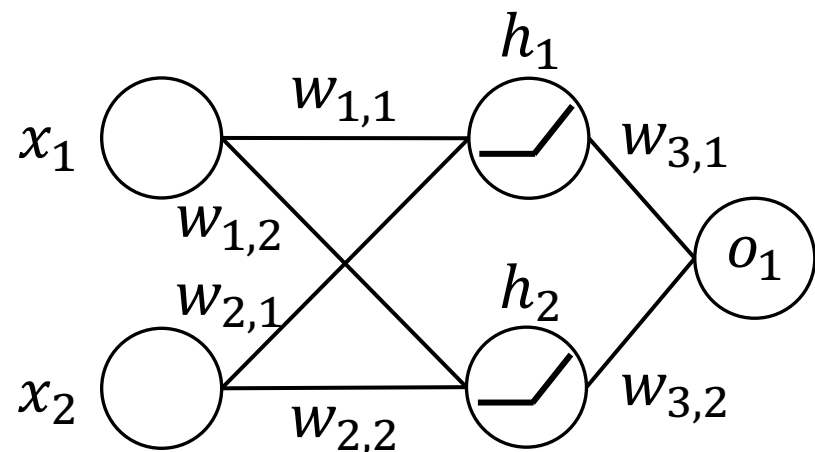
相當於隱藏層+非線性轉換
為原始資料點帶來兩條分割線



Initial values $\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$ Final values $\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$



上一頁的計算過程



x1	x2	XOR	o_1
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

$$o_1 = h_1 w_{3,1} + h_2 w_{3,2} + b_3$$

$$= \text{ReLU}(x_1 w_{1,1} + x_2 w_{2,1} + b_1) w_{3,1} + \text{ReLU}(x_1 w_{1,2} + x_2 w_{2,2} + b_2) w_{3,2} + b_3$$

1
1
0
1
1
1
-1
-2
0



Summary (for XOR with MLP)

- XOR 在輸入空間中 線性不可分，一般線性分類器無法解決
- 非線性轉換 (ReLU) 創造隱藏空間，讓資料從不可分 → 可分
 - 在隱藏空間中，XOR 點已經 線性可分
 - 輸出層只需一條線 (線性分類器) 就能完成分類
- MLP 的力量：線性 + 非線性堆疊 → 能解決更複雜的決策邊界



XOR：隱藏層如何把資料掰開

同一個平面，被線性層壓扁、再被 ReLU 折起來

$$h = \text{ReLU}(W^{\top}x + c)$$

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$y = w^{\top}h + b, \quad w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0$$



More resources for XOR

- Community blog post
 - <https://dev.to/jbahire/demystifying-the-xor-problem-1blk>
 - <https://chih-sheng-huang821.medium.com/機器學習-神經網路-多層感知機-multilayer-perceptron-mlp-運作方式-f0e108e8b9af>
- Stanford course:
 - https://youtu.be/s7nRWh_3BtA?si=ZEGRN5pmPchM0NxM
- Also, in Deep Learning book (Chapter 6.1)



Gradient Descent

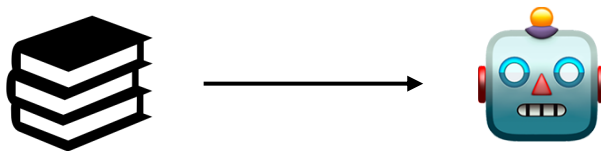
Machine Learning Approaches

Approaches

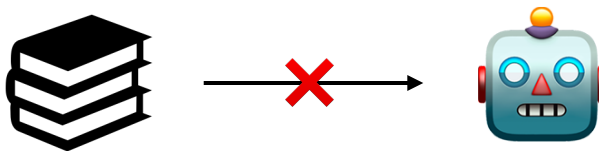
Supervised
Learning
(監督式學習)

Unsupervised
Learning
(無監督式學習)

Learning Process



準備許多有答案 (label) 的資料來
訓練模型



模型在沒有標註的資料下學習資料
本身的內在結構或模式

Common Tasks

Classification

Regression

Clustering

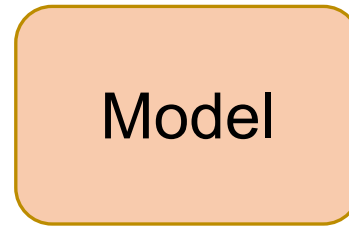
Similarity
Matching

Dimensionality
Reduction



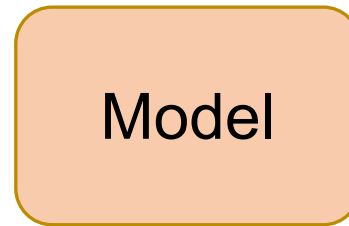
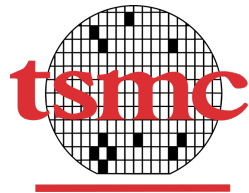
Classification and Regression

Classification



Cat ☒
Dog
Apple

Regression



1100



[Example] Clustering

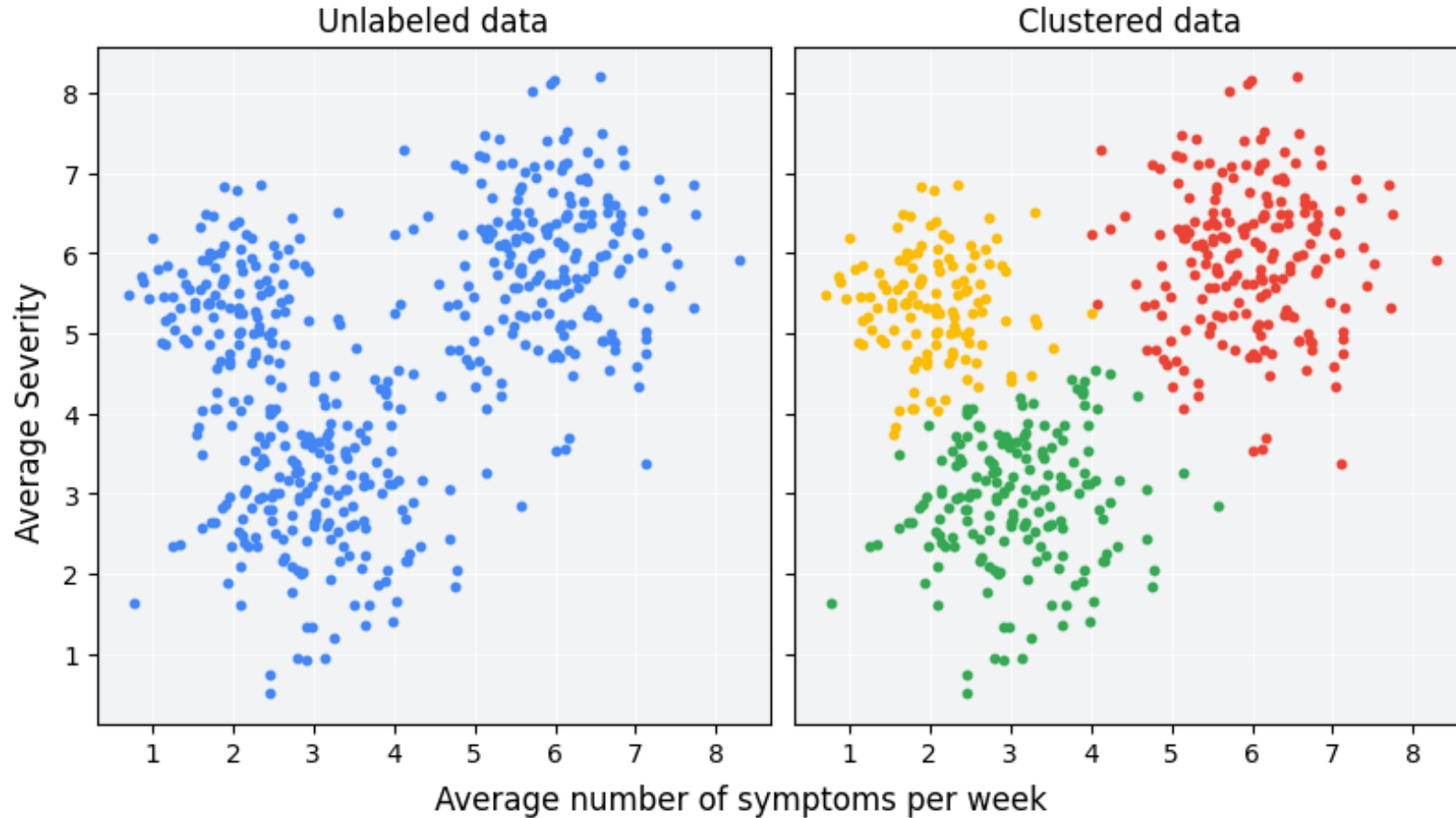


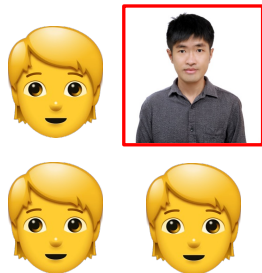
Figure source: <https://developers.google.com/machine-learning/clustering/overview?hl=zh-tw>



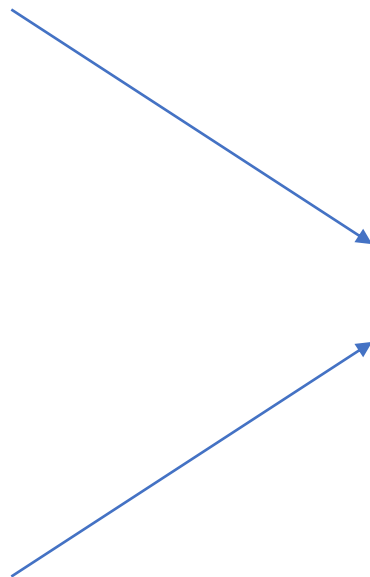
[Example] Similarity Matching

以人臉辨識為例

資料庫



我被掃描的照片



Similarity
Matching

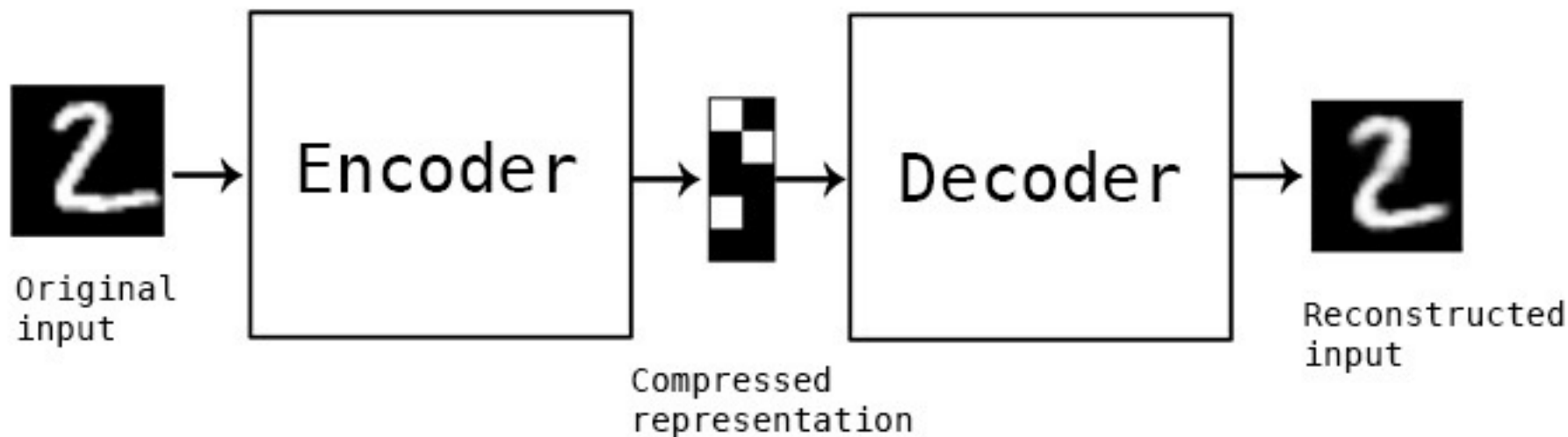


Valid or
Invalid



[Example] Dimensionality Reduction

以 AutoEncoder 為例



類似於近年的 self-supervised learning

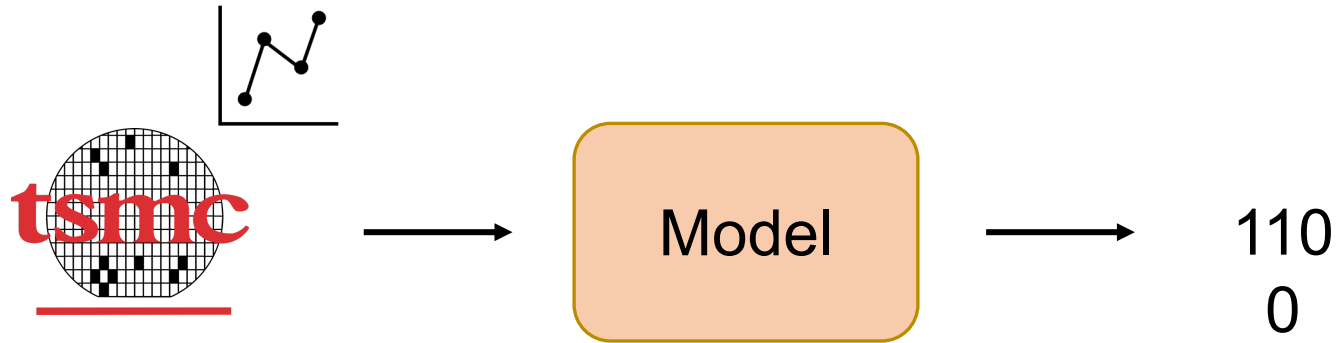


Figure source: https://blog.keras.io/img/ae/autoencoder_schema.jpg

Regression

Regression Tasks (1)

Regression



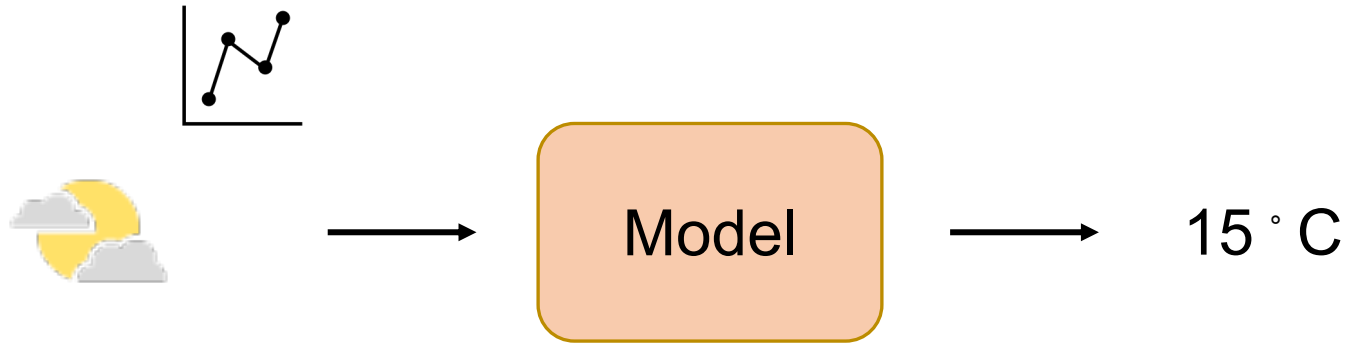
2/17	2/18	2/19	2/20	2/21
1080	1085	1095	1080	?

Prediction based on the past data (Time-series)



Regression Tasks (2)

Regression



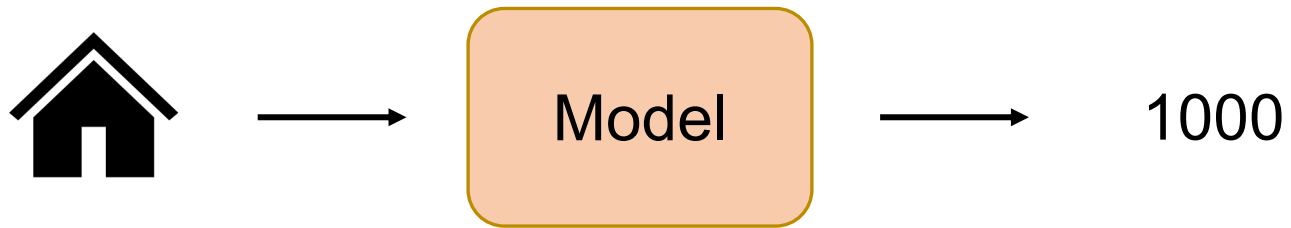
2/17	2/18	2/19	2/20	2/21
15 ° C	14 ° C	13 ° C	15 ° C	?

Prediction based on the past data (Time-series)



Regression Tasks (3)

Regression

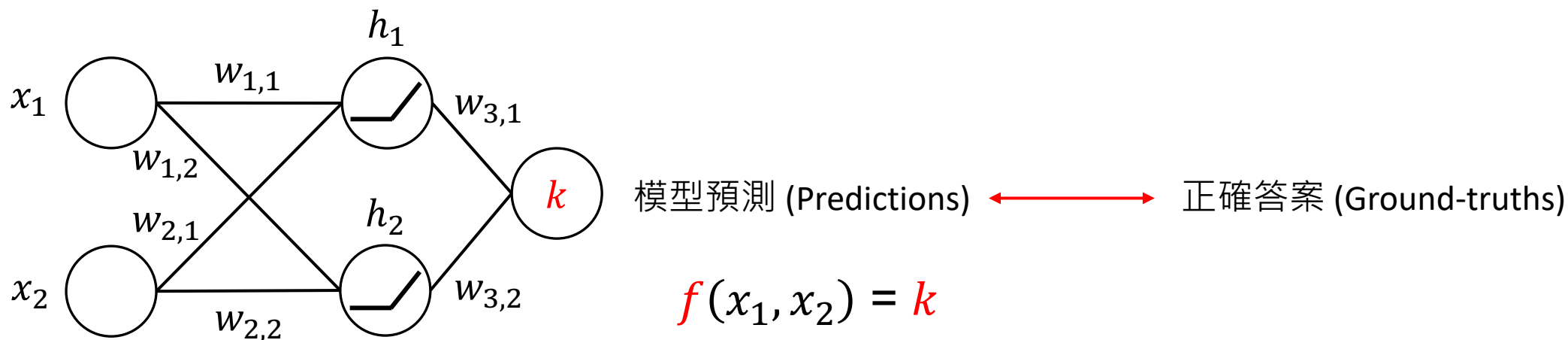


Area	City	Num of Rooms	House Age	Price
15	Taoyuan	3	10	?

Prediction based on the different features (Tabular data)



Machine Learning is to train a function

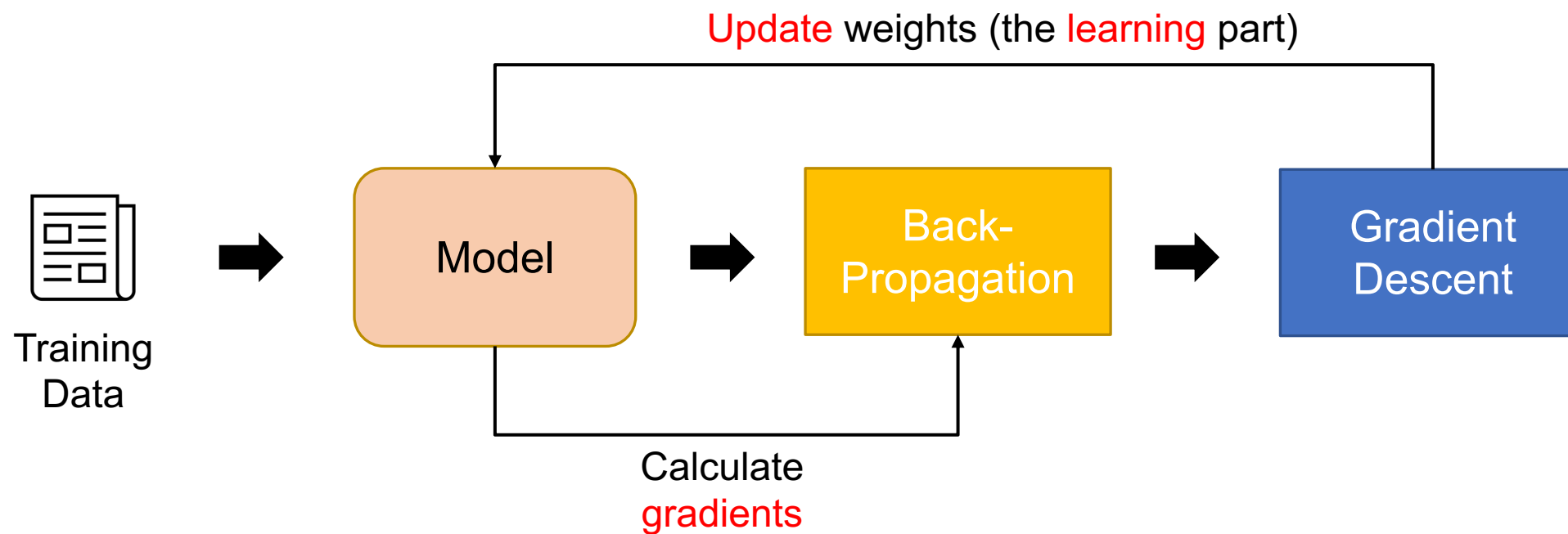


- 💡 透過 **Back-Propagation** 計算神經網路中的梯度
- 📈 透過 **Gradient Descent** (梯度下降) 更新權重值 (w 跟 b)



Training Process of a Deep Learning Model

- 深度學習模型被訓練的流程

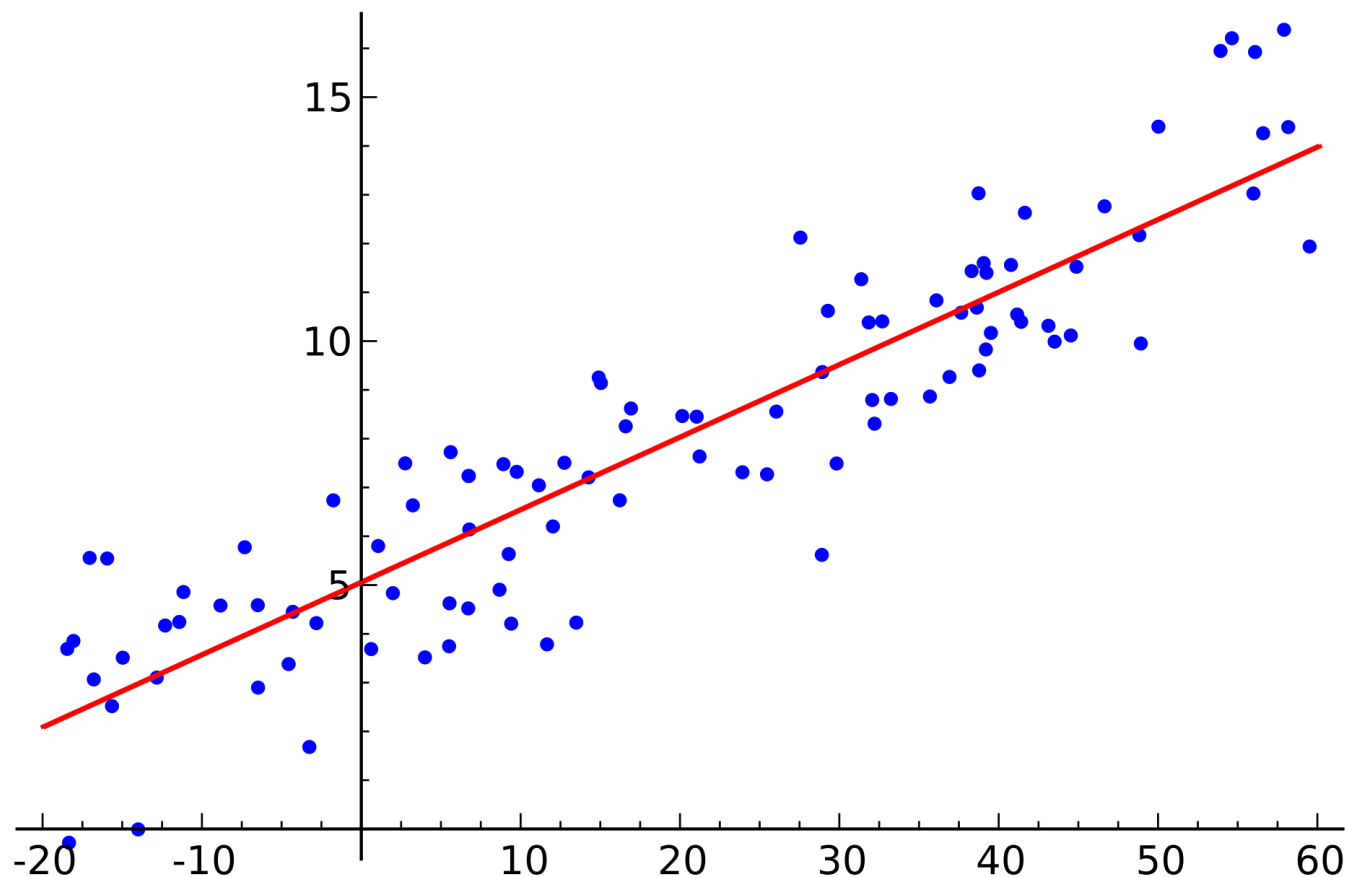


Gradient Descent vs. Back-propagation

	Gradient Descent	Back-propagation
功能	根據梯度更新權重值 (weights)	計算神經網路中的梯度，以供梯度下降使用
最終目標	透過梯度找出 function 最佳解 (最小化目標函數)	計算梯度，以便使用梯度下降找到最佳解



Linear Regression (線性迴歸)



紅線為直線方程式 $y = wx + b$

Linear regression 通常透過均方誤差來擬合資料

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

紅線代表可以使所有藍色點的MSE最小的w跟b代入直線方程式後得到的線



為什麼Regression叫做Regression？

起源 [編輯]

回歸的最早形式是**最小平方法**，由1805年的**勒壤得**（Legendre）^[1]，和1809年的**高斯**（Gauss）出版^[2]。勒壤得和高斯都將該方法應用於從天文觀測中確定關於太陽的物體的軌道（主要是彗星，但後來是新發現的小行星）的問題。高斯在1821年發表了最小平方理論的進一步發展^[3]，包括**高斯－馬可夫定理**的一個版本。

「迴歸」一詞最早由**法蘭西斯·高爾頓**（Francis Galton）所使用^{[4][5]}。他曾對親子間的身高做研究，發現父母的身高雖然會遺傳給子女，但子女的身高卻有逐漸「回歸到中等（即人的**平均值**）」的現象。不過現在的迴歸已經和當初的意義不盡相同。

在1950年代和60年代，經濟學家使用機械電子桌面計算器來計算回歸。在1970年之前，這種計算方法有時需要長達24小時才能得出結果^[6]。

<https://zh.wikipedia.org/zh-tw/迴歸分析>

<https://stats.stackexchange.com/questions/11087/why-are-regression-problems-called-regression-problems>



Math Warning!!

What are gradients? (數學定義)

- First-order derivative (s)
 - Univariate function: a scalar
 - Multivariate function: a vector
- **Univariate** Example:

Original function: $f(x) = x^2$

First-order derivative: $f'(x) = 2x = \nabla_x f$



What are gradients? (數學定義)

- First-order derivative (s)
 - Univariate function: a scalar
 - Multivariate function: a vector
- **Multivariate** Example:

Original function: $f(x, y) = x^2 + y^2$

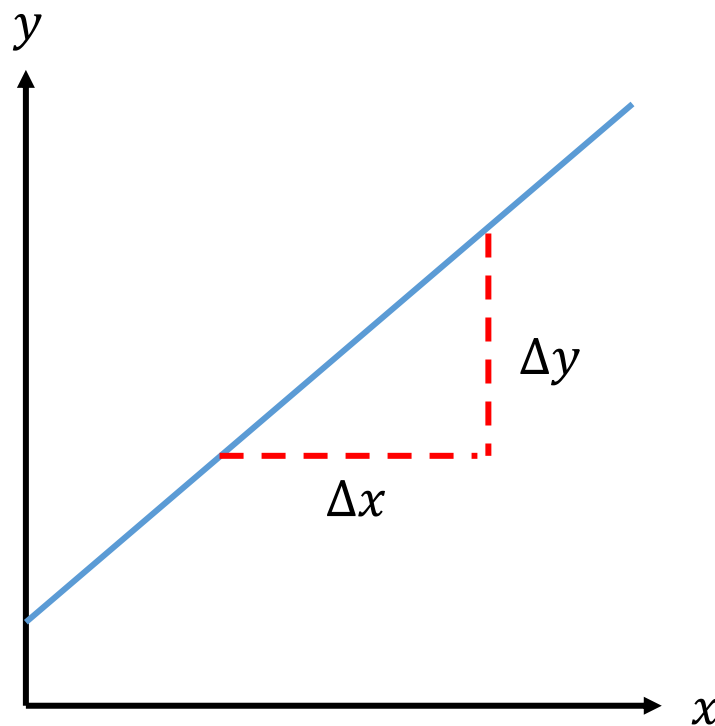
First-order derivatives: $f'(x, y) = \begin{bmatrix} 2x \\ 2y \end{bmatrix} = \nabla_{x,y} f$



What are gradients? (數學意義)

$$y = mx + b$$

$$\text{Slope (斜率)} = m$$



$$\text{斜率} = \frac{\Delta y}{\Delta x}$$

$$= \frac{y_2 - y_1}{x_2 - x_1}$$

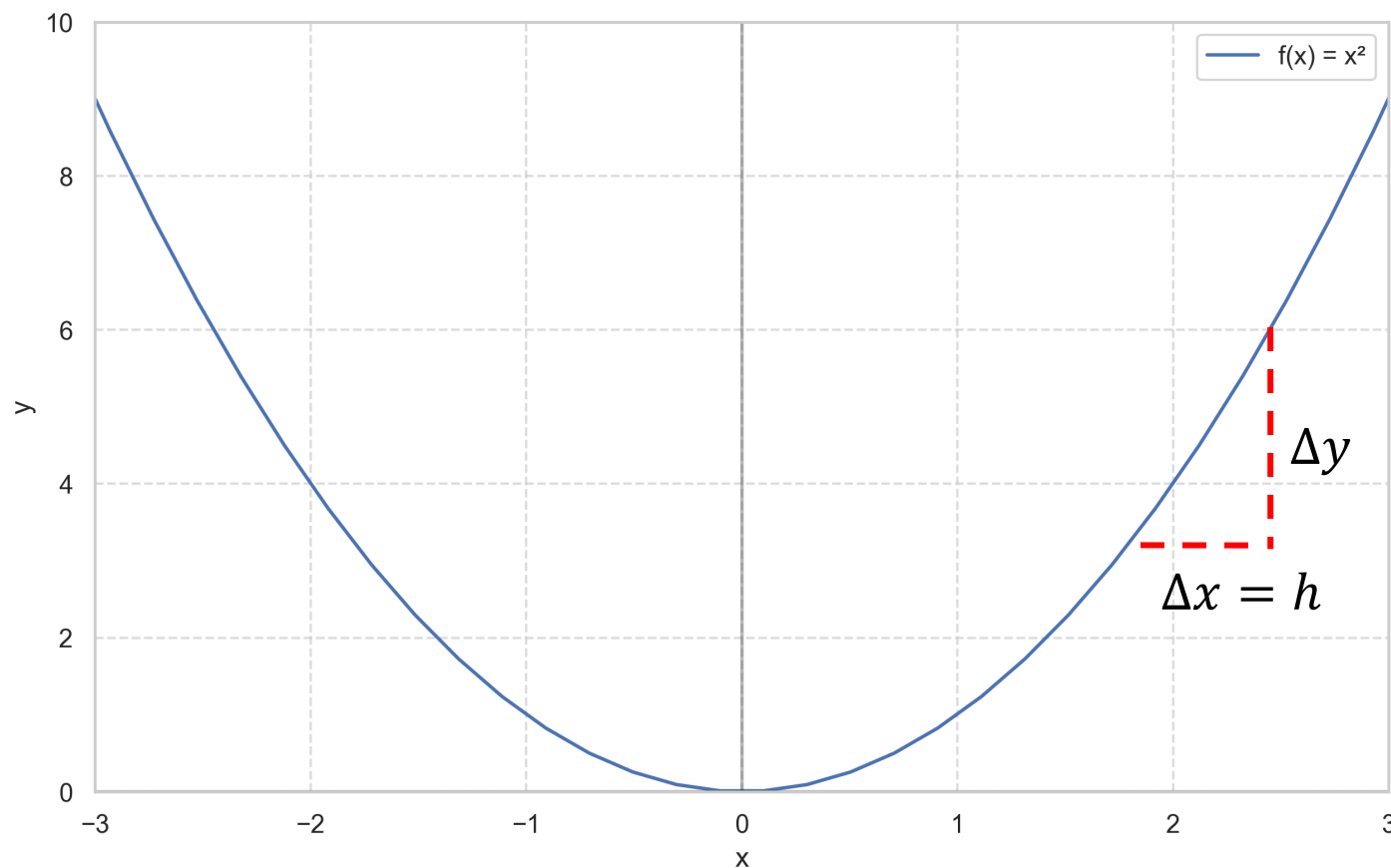
$$= \frac{(mx_2 + b) - (mx_1 + b)}{x_2 - x_1}$$

$$= \frac{m(x_2 - x_1)}{x_2 - x_1}$$

$$= m$$



What are gradients? (數學意義)



$$\text{斜率} = \frac{\Delta y}{\Delta x}$$

$$f(x) = x^2$$

一次導函數：

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$= 2x$$

梯度等於一次導函數
的值，也就是斜率



計算導數

$$f(x) = x^2$$

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

$$= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$

$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$

$$= \lim_{h \rightarrow 0} 2x + h = 2x \quad (h \text{ 趨近於 } 0)$$



What are gradients? (物理意義)

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- 有一跑者10分鐘內跑了2km，請問這個人一秒鐘跑多少公里？
- 速度 = 距離 / 時間
- 一秒鐘可以被想成h趨近於0
- 因此，梯度代表在極短情況 (這裡用時間舉例) 的變化量



Gradient Descent (梯度下降法)

Assume x is a **trainable** parameter (**weight**), f is a **differentiable** function:

Gradient descent:
$$x' = x - \eta \nabla_x f(x)$$

Objective function
(example):
$$f(x) = \frac{1}{2}x^2$$

η is the **learning rate** used for gradient descent.



What “Function” we’re going to train?

- Loss Function
- 以均方誤差 (Mean Squared Error) 為例：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

我們要訓練的 function

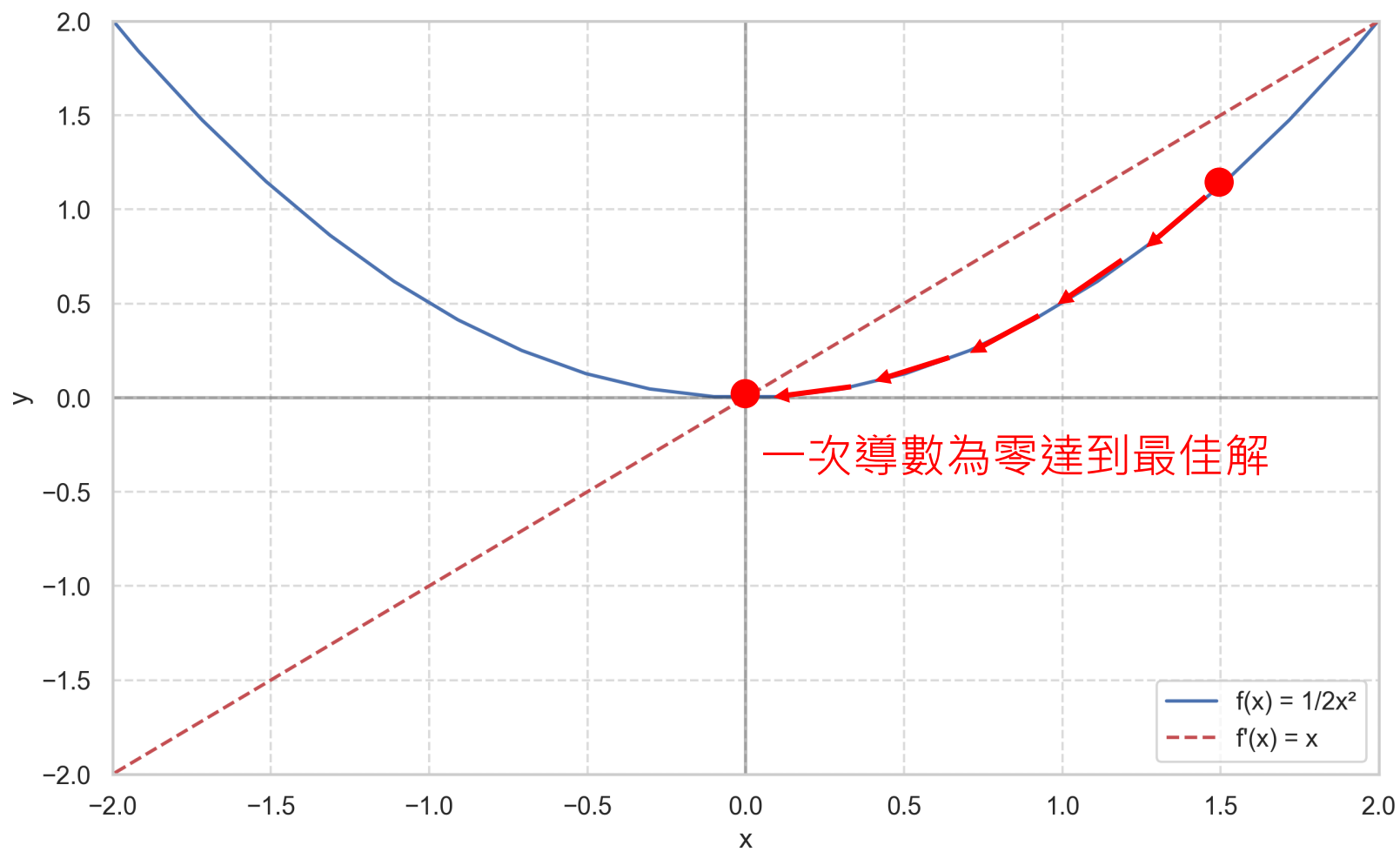
$$= \frac{1}{n} \sum_{i=1}^n (y_i - \underline{(wx_i + b)})^2$$

假設模型的輸出是 $wx_i + b$

- 其中:
 - n 代表有 n 筆訓練資料
 - y_i 為任一筆 ground-truth、 $\hat{y}_i$ 為任一筆 prediction (model output)

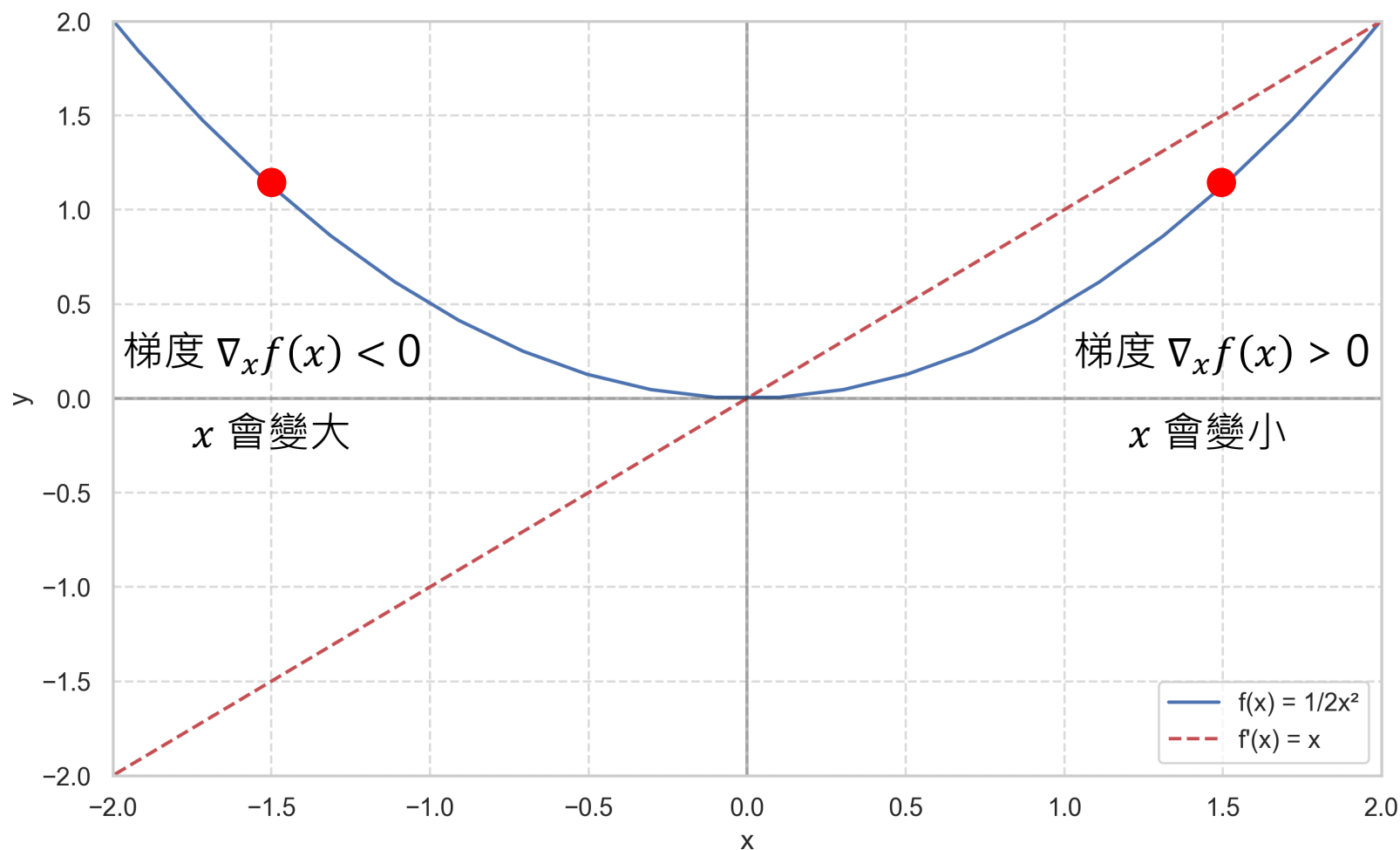


Gradient Descent



以兩個點來觀察 Gradient Descent 的特性

$$x' = x - \eta \nabla_x f(x)$$



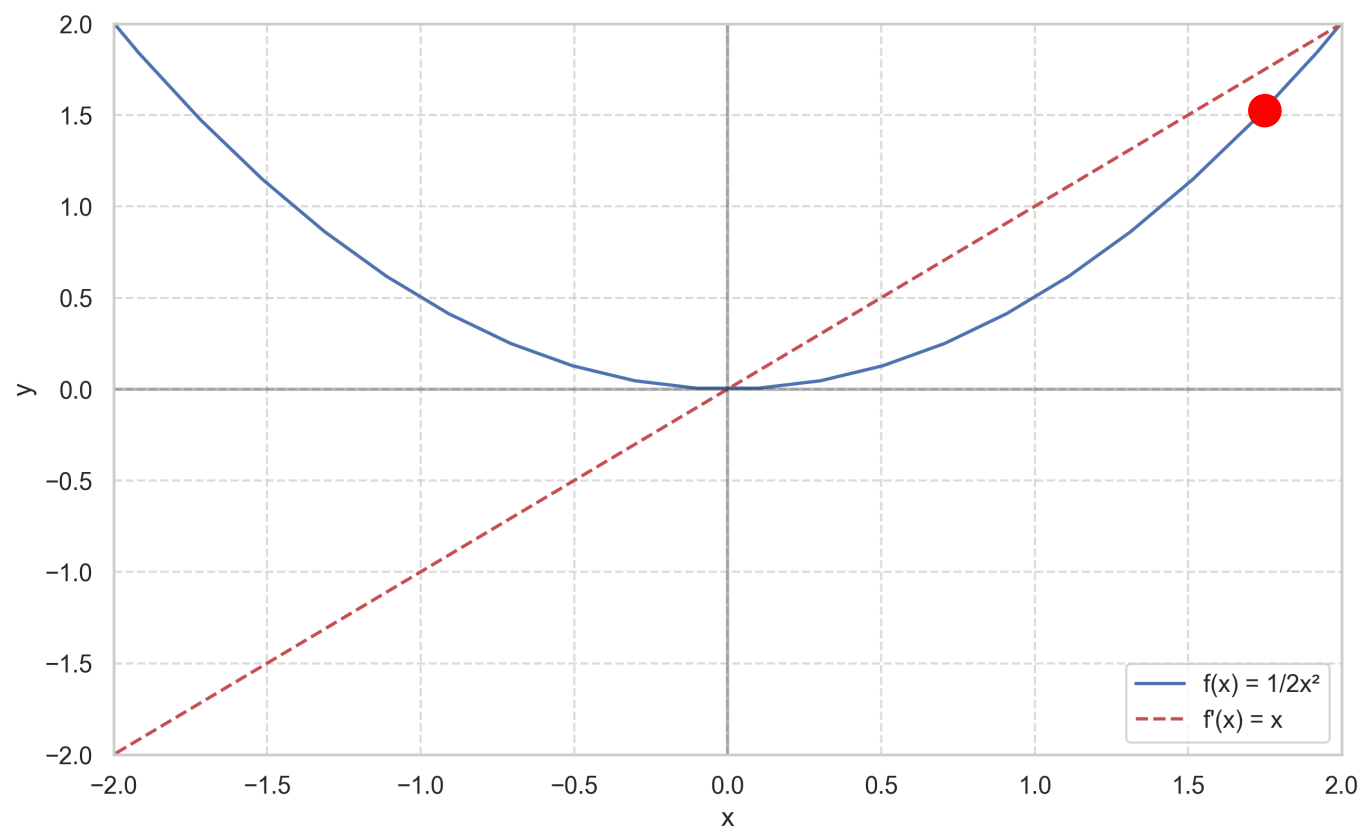
為什麼可以用位置減梯度？(1/3)

- 梯度主要是提供參數要修正的「方向」
 - 正號或負號
- 讓參數的值改變最後順利走到最低點



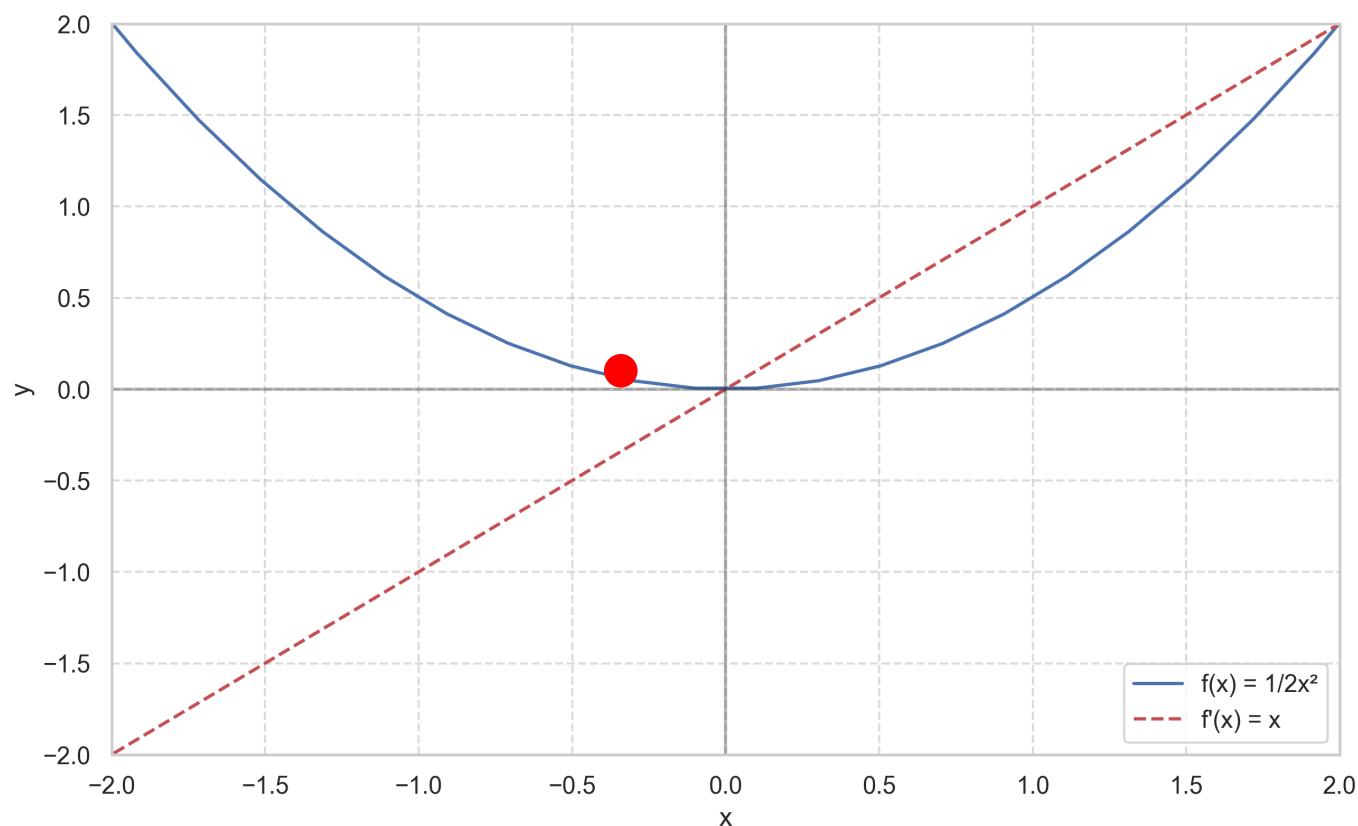
為什麼可以用位置減梯度？(2/3)

- 如果|梯度|很大的話，表示該位置還在非常陡峭的位置，**通常**在這時候參數的值確實需要改變多一點



為什麼可以用位置減梯度？(3/3)

- 如果|梯度|很小的話，表示該位置在函數平滑的位置，**通常**在這時候參數的值可能需要改變少一點





Gradient Descent 流程推導

Minimize a Regression Model

- 以均方誤差 (Mean Squared Error) 為例：

$$\begin{aligned}\mathcal{L} &= \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2\end{aligned}$$

- 其中:
 - $\mathcal{L}$ 代表 **Loss function** ; n 代表有 n 筆訓練資料
 - y_i 為任一筆 ground-truth 、 $\hat{y}_i$ 為任一筆 prediction (model output)



Minimize a Regression Model

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

對 w 進行偏微分

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-x_i)$$

對 b 進行偏微分

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-1)$$



The generalized power rule for derivatives

Assume $y = [u(x)]^n$, and $u(x)$ is **differentiable**.

Then $\frac{dy}{dx} = n \cdot [u(x)]^{n-1} \cdot u'(x)$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-x_i)$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-1)$$



Minimize a Regression Model

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-x_i)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(y_i - (wx_i + b)) \cdot (-1)$$

更新 w :

$$w_t = w_{t-1} - \eta \frac{\partial \mathcal{L}}{\partial w_{t-1}}$$

更新 b :

$$b_t = b_{t-1} - \eta \frac{\partial \mathcal{L}}{\partial b_{t-1}}$$

- η 代表 **learning rate** , 扮演梯度下降過程的重要角色



The impact of a Learning Rate

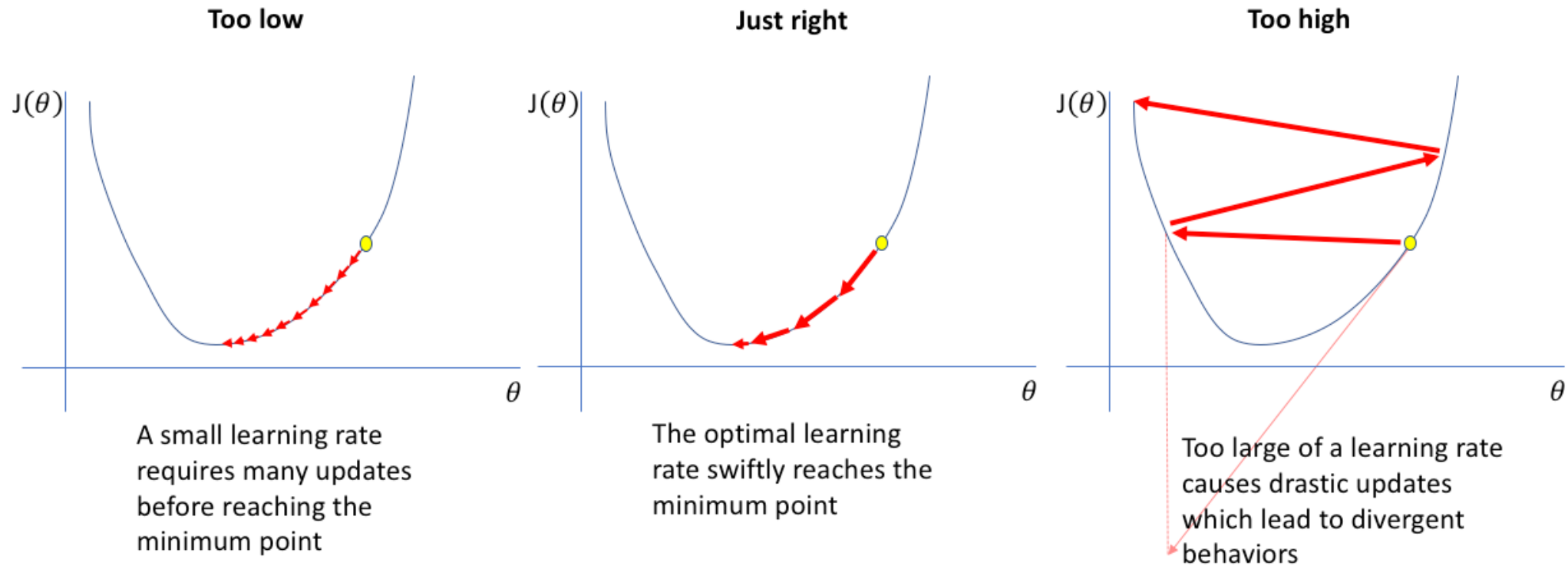


Figure source: <https://www.jeremyjordan.me/nn-learning-rate/>



How to choose the best learning rate?



Figure source:
<https://x.com/karpathy/status/801621764144971776>

More resources for Calculus

<https://calcgospel.top/wp-content/uploads/2020/06/02-01-微分的定義.pdf>



More resources for gradient descent

Deep Learning Book Chapter 4.3: Gradient-Based Optimization

- <https://www.deeplearningbook.org/contents/numerical.html>

<https://www.youtube.com/watch?v=fegAeph9UaA>

[https://speech.ee.ntu.edu.tw/~tlkagk/courses/ML_2016/Lecture/Regression%20\(v6\).pdf](https://speech.ee.ntu.edu.tw/~tlkagk/courses/ML_2016/Lecture/Regression%20(v6).pdf)



Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw